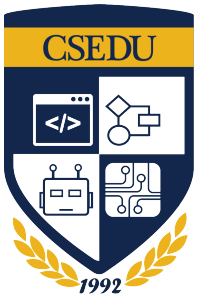

University of Dhaka

Department of Computer Science & Engineering

CSE 3216 - Software Design Patterns Lab

Project Report

Project Name: Filr Chrome Extension

Team Name: DU_xBlitz

Submitted By

Nafis Shyan - 10

Anirban Roy Sourov - 32

Mominul Islam Hemal - 40

28th Batch
Department of Computer Science & Engineering
University of Dhaka

Submitted On
27 December 2025

Contents

1 Introduction

Filr is a sophisticated Chrome browser extension designed to streamline the process of filling out government forms by automating document extraction and form population. The extension leverages artificial intelligence to extract information from various government-issued documents (such as birth certificates, National ID cards, passports, utility bills, and education certificates) and automatically fills web forms with the extracted data, significantly reducing manual data entry time and minimizing errors.

1.1 Problem Definition & Context

Filling out government forms in Bangladesh is a time-consuming and error-prone process. Citizens often need to:

- Manually extract information from multiple physical documents (birth certificates, NID cards, utility bills, etc.)
- Transcribe this information into web-based government forms, often dealing with bilingual fields (English and Bengali)
- Handle complex date formats, address formats, and field mappings
- Repeat this process for multiple forms, leading to redundant data entry

The problem is compounded by several factors:

- **Form Complexity:** Government forms often have numerous fields requiring specific formatting and validation
- **Language Barriers:** Forms contain both English and Bengali text, requiring users to switch between languages
- **Document Variety:** Different forms require different combinations of documents, making it difficult to know which documents to prepare
- **Error-Prone Process:** Manual transcription leads to typos, formatting errors, and data inconsistencies
- **Time Consumption:** A single form can take 15-30 minutes to complete manually

Filr addresses these challenges by providing an intelligent, automated solution that:

- Automatically detects form fields on any government website
- Identifies which documents are required for a specific form
- Extracts structured data from uploaded documents using AI
- Intelligently maps extracted data to form fields
- Automatically fills forms with proper formatting and validation
- Works offline with automatic synchronization when connectivity is restored

1.2 Motivation

The motivation behind developing Filr stems from several key observations and needs:

Public Service Efficiency Government digitalization initiatives aim to make services more accessible, but the complexity of form-filling remains a barrier. By automating this process, Filr contributes to making government services more user-friendly and accessible to all citizens, regardless of their technical proficiency.

Time and Resource Savings Manual form filling consumes significant time for both citizens and government employees who must verify and process applications. Automating this process saves hours of work per application, allowing citizens to focus on other important tasks and enabling government offices to process applications more efficiently.

Error Reduction Manual data entry is inherently error-prone. Typos, formatting mistakes, and data inconsistencies can lead to application rejections, requiring citizens to resubmit forms. Filr's automated extraction and filling process significantly reduces such errors by ensuring consistent, accurate data entry.

Accessibility and Inclusion Not all citizens are comfortable with technology or bilingual data entry. Filr democratizes access to digital government services by handling the technical complexity automatically, making services accessible to a broader population.

Educational Value This project serves as an excellent learning opportunity to implement multiple software design patterns (MVC, Chain of Responsibility, Factory, Observer, Singleton, Decorator, Strategy, Prototype, Command, State, Mediator, and Proxy patterns) in a real-world application, demonstrating how design patterns solve practical problems while maintaining code quality and extensibility.

1.3 Core Features

Filr provides a comprehensive set of features designed to address the complete form-filling workflow:

Intelligent Form Detection The extension automatically analyzes web pages to detect form structures, extract field metadata (labels, IDs, names), and identify the type of form being filled. This feature uses AI to understand form context and structure, eliminating the need for manual form configuration.

Dynamic Document Requirement Analysis Based on detected form fields, Filr intelligently determines which source documents are required to fill the form. The system provides a clear list of required documents in both English and Bengali, along with confidence levels and necessity indicators (required, optional, conditional).

Multi-Document Processing Filr supports processing multiple document types through a flexible processor chain:

- Birth Certificates
- National ID Cards
- Passports
- Education Certificates
- Utility Bills

Each document type has a specialized processor that extracts relevant information using AI-powered analysis.

Dual Extraction Modes

- **Static Extraction:** Uses predefined schemas for known document types, providing consistent extraction for standard documents
- **Dynamic Extraction:** Builds extraction schemas dynamically based on detected form fields, allowing the system to adapt to any form structure

Intelligent Form Filling The extension automatically maps extracted data to form fields using multiple matching strategies:

- Exact field name/ID matching
- Fuzzy matching for variations
- Intelligent handling of date fields (splitting into day/month/year components)
- Special handling for dropdown menus and select fields
- Event triggering for framework compatibility (React, Vue, etc.)

Offline Processing & Synchronization Filr operates seamlessly in offline environments:

- Automatic network status detection
- Local document queueing using IndexedDB
- Automatic synchronization when connectivity is restored
- Retry logic with exponential backoff for failed operations
- Queue management interface for monitoring and controlling queued jobs

User-Friendly Interface

- Modern React-based side panel interface
- Real-time progress indicators
- Toast notifications for user feedback
- Queue status display
- Network status indicators
- Results review page with editable fields

Data Security & Privacy

- All processing occurs client-side where possible
- API keys stored securely in browser storage
- Documents processed through secure API endpoints
- No data stored on external servers
- Local caching for improved performance

1.4 Tools, Technologies & Frameworks Used

Filr is built using a modern, robust technology stack that ensures performance, maintainability, and extensibility:

Frontend Framework & Libraries

- **React 19.1.1:** Modern UI framework for building the extension's user interface
- **TypeScript 5.9.2:** Type-safe JavaScript for improved code quality and developer experience
- **React Hook Form 7.66.0:** Form state management and validation
- **Zod 3.24.1:** Schema validation library for runtime type checking

UI Components & Styling

- **Tailwind CSS 4.1.14:** Utility-first CSS framework for rapid UI development
- **shadcn/ui:** High-quality, accessible React component library built on Radix UI primitives
- **Radix UI:** Unstyled, accessible component primitives (Dialog, Select, Tabs, Tooltip, etc.)
- **Lucide React:** Icon library for modern, consistent iconography

Browser Extension Framework

- **WXT 0.20.6:** Modern framework for building browser extensions with TypeScript support
- **Chrome Extension Manifest V3:** Latest extension manifest format for Chrome browsers
- **Background Service Worker:** For background processing and network monitoring
- **Content Scripts:** For interacting with web page DOM
- **Side Panel API:** For the extension's main user interface

AI & Machine Learning

- **Google Gemini AI API (@google/genai 1.28.0):** For document extraction and form analysis
- **Structured Output:** Using Gemini's schema-based extraction for reliable data parsing
- **Multi-modal Processing:** Support for PDF and image document formats

Data Storage & Management

- **IndexedDB:** For offline document queue storage and job management
- **localStorage:** For settings, API keys, and form cache
- **Chrome Storage API:** For extension-specific data persistence

Development Tools

- **Node.js & npm:** Package management and build tooling
- **PostCSS & Autoprefixer:** CSS processing and vendor prefixing
- **TypeScript Compiler:** Type checking and compilation

Design Patterns Implementation The project implements multiple design patterns using TypeScript:

- **MVC Pattern:** Separation of Models, Views, and Controllers
- **Chain of Responsibility:** Document processor chain
- **Factory Pattern:** Processor creation and chain building
- **Observer Pattern:** Progress notifications and event handling
- **Singleton Pattern:** API client and service management
- **Decorator Pattern:** Processor enhancement (logging, metrics, rate limiting)
- **Strategy Pattern:** Extraction strategy selection
- **Prototype Pattern:** Document structure cloning

- **Command Pattern:** Operation encapsulation
- **State Pattern:** Application state management
- **Mediator Pattern:** Component communication
- **Proxy Pattern:** Request routing and offline handling

1.5 Individual Responsibilities

The Filr project was developed collaboratively by a team of three members, with each member contributing expertise in different areas. The following responsibilities are based on actual git commit history and code contributions:

Nafis Shyan (Roll 10)

- **Design Patterns Refactoring:** Refactored the codebase implementing multiple design patterns including Chain of Responsibility, MVC, Decorator, Singleton, Mediator, Command, Strategy, and Prototype patterns
- **Form Detection & Extraction:** Implemented dynamic form detection, data extraction, and form fillup functionality
- **Document Processing Pipeline:** Developed the processor chain system and individual document processors for various document types
- **Offline Functionality:** Implemented complete offline functionality codebase with caching mechanisms
- **Session Management:** Added new caching mechanism for session management to prevent document loss during navigation
- **Bug Fixes:** Fixed critical bugs related to document processing and pattern implementations
- **Architecture Documentation:** Created comprehensive architecture documentation explaining design pattern implementations

Anirban Roy Sourov (Roll 32)

- **Offline Features:** Implemented offline data encryption with user-chosen password and offline save option as TOON
- **Secure Storage:** Migrated to secure Chrome storage API for sensitive data management
- **Toon Import Functionality:** Developed complete toon import facility with file extension support, upload functionality, and confirmation page
- **UI Components:** Added "Detect Form" and "Update info" options at homepage
- **Local Storage Management:** Fixed "update info" functionality to save data locally in localStorage instead of navigating to another page
- **UI Fixes:** Fixed update info button alignment and upper layout visibility issues
- **File Upload:** Fixed issues related to file upload and Gemini processing
- **Initial Project Setup:** Contributed to initial project push before pattern-based refactoring

Mominul Islam Hemal (Roll 40)

- **Project Initialization:** Set up the initial WXT React project structure and configuration
- **UI Framework Setup:** Configured Tailwind CSS and ShadCN UI components for the extension
- **Sidepanel Mode:** Initialized and optimized Chrome extension sidepanel mode for better user experience
- **Form Filling Implementation:** Developed dynamic form filler with ref_id support and improved form fillup logic
- **Accessibility Tree:** Switched from full HTML extraction to accessibility tree for better form detection accuracy
- **UI/UX Improvements:** Redesigned UploadPage for mobile-first sidepanel experience with step-by-step progress indicators
- **Page Optimization:** Optimized entry and settings pages specifically for Chrome extension sidepanel
- **Home Page Enhancement:** Improved home page UX by removing queue status, enhancing styling, and updating copy
- **Progress Tracking:** Implemented progress tracking functionality for form detection workflow
- **Button Interactions:** Fixed button click handlers and improved user interaction flows

Collaborative Efforts All team members contributed to:

- Code reviews and quality assurance
- Bug fixes and performance optimization
- Project planning and requirement analysis
- Testing and validation of features
- Documentation and report writing

2 System Overview

This section provides a comprehensive overview of the Filr extension's system architecture and use cases, illustrating how the various components interact to deliver the complete form-filling solution.

2.1 System Architecture

The Filr extension follows a modular, layered architecture that separates concerns across multiple tiers. This architecture ensures maintainability, scalability, and clear separation between presentation, business logic, and data access layers.

2.1.1 High-Level Architecture

The system is organized into four primary layers, each with distinct responsibilities:

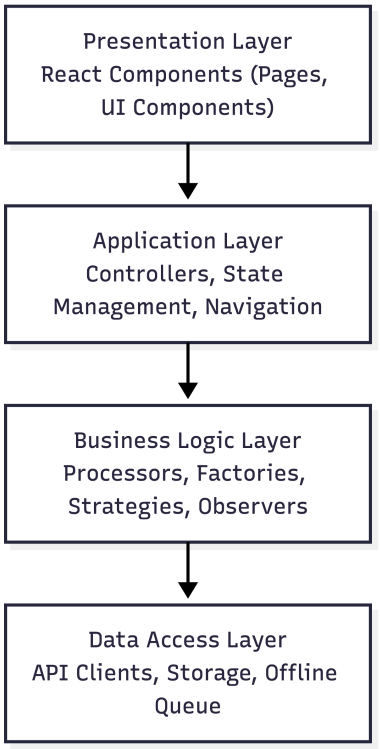

Figure 1: Layered Architecture of Filr Extension

Presentation Layer The topmost layer consists of React-based user interface components that provide the user-facing functionality:

- **Pages:** FormDetectionPage, UploadPage, ResultsPage, SettingsPage, TraditionalFormPage
- **UI Components:** Reusable components from shadcn/ui library (buttons, dialogs, forms, etc.)
- **Status Indicators:** NetworkStatusBanner, OfflineStatusIndicator, QueueStatus

Application Layer This layer orchestrates user interactions and manages application state:

- **Controllers:** DocumentController, OfflineAwareDocumentController
- **State Management:** AppState using State pattern for navigation and context management
- **Navigation:** Page routing and state transitions

Business Logic Layer The core processing logic resides in this layer:

- **Processors:** DocumentProcessor chain (BirthCertificate, NID, Passport, Education, UtilityBill)
- **Factories:** ProcessorFactory for creating processor chains
- **Strategies:** ExtractionStrategy for static vs. dynamic extraction
- **Observers:** ProcessingObserver for progress notifications
- **Other Patterns:** Decorators, Singletons, Prototypes, Commands

Data Access Layer The bottom layer handles all data persistence and external communication:

- **API Clients:** Gemini AI API integration (APIClientSingleton)
- **Storage:** IndexedDB for offline queue, localStorage for settings
- **Offline Queue:** SimpleOfflineQueue, LocalDocumentStore, SyncManager
- **Network Management:** NetworkStatusManager for connectivity monitoring

2.1.2 Component Interaction Architecture

The extension consists of three main runtime components that communicate through message passing:

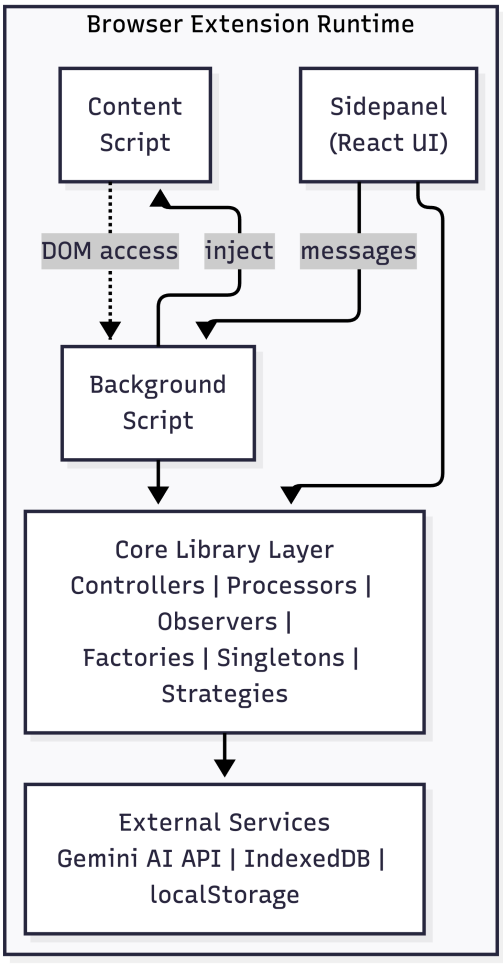

Figure 2: Component Interaction Architecture

Sidepanel Component The React-based user interface runs in Chrome's side panel, providing:

- Form detection interface
- Document upload interface
- Results display and editing
- Settings management
- Queue status monitoring

Background Script The service worker handles:

- Network status monitoring
- Message passing between components
- Background processing coordination
- Storage management

Content Script Injected into web pages to:

- Extract HTML source for form detection
- Inject form-filling scripts
- Interact with page DOM elements
- Trigger form field events

2.1.3 Data Flow Architecture

The system processes data through several key flows:

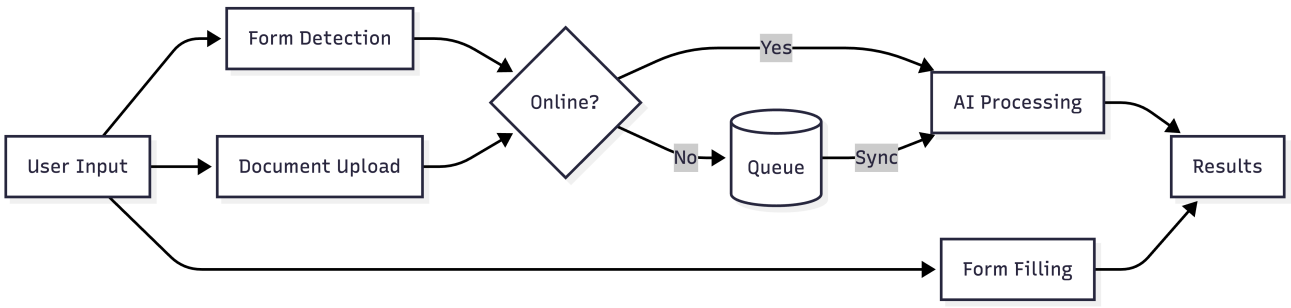

Figure 3: High-Level Data Flow

2.2 Use Case Diagrams

Use case diagrams illustrate the interactions between users (actors) and the system, showing the various functionalities that Filr provides.

2.2.1 Primary Use Cases

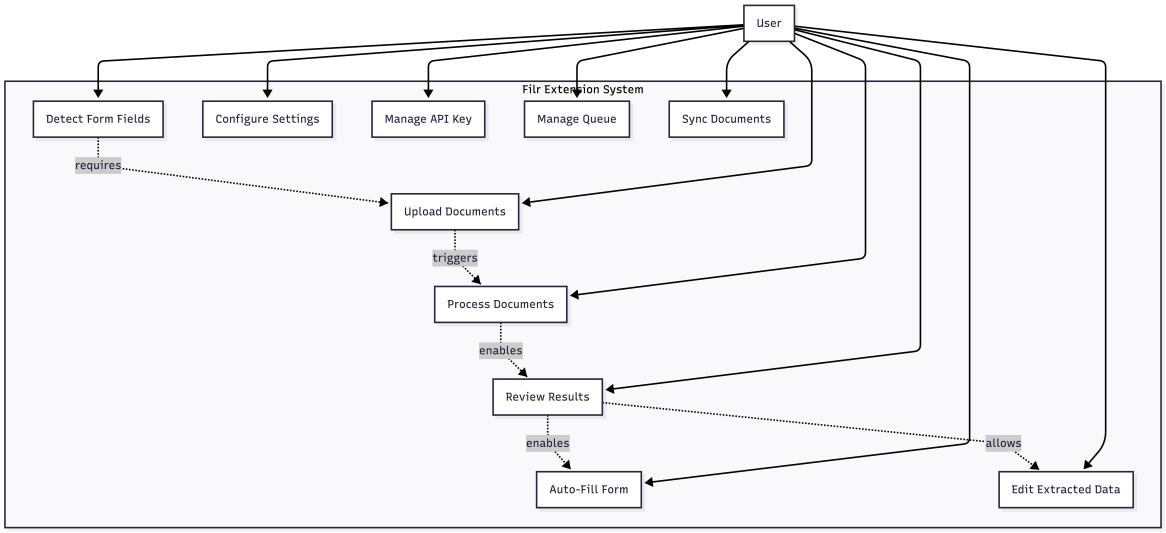

Figure 4: Primary Use Case Diagram

Form Detection Use Cases

- **Detect Form Fields:** User navigates to a government form page and triggers form detection. The system analyzes the page HTML and extracts form structure, field names, IDs, and labels.
- **Identify Required Documents:** Based on detected form fields, the system determines which source documents are needed to fill the form.

Document Processing Use Cases

- **Upload Documents:** User selects and uploads document files (PDF, JPG, PNG) corresponding to the required documents.
- **Process Documents:** The system processes uploaded documents through the processor chain, extracting structured data using AI.
- **Review Results:** User reviews the extracted data in a structured format, with fields organized by document type.
- **Edit Extracted Data:** User can manually correct any errors in the extracted data before form filling.

Form Filling Use Cases

- **Auto-Fill Form:** User triggers automatic form filling. The system maps extracted data to form fields and populates them with proper formatting and validation.

Configuration Use Cases

- **Configure Settings:** User manages extension settings such as preferred AI model, extraction preferences, and UI preferences.
- **Manage API Key:** User enters and stores their Google Gemini API key securely.

Offline Processing Use Cases

- **Manage Queue:** User views, retries, or cancels queued documents when offline.
- **Sync Documents:** When connectivity is restored, the system automatically processes queued documents.

2.2.2 Extended Use Case Diagram with System Actors

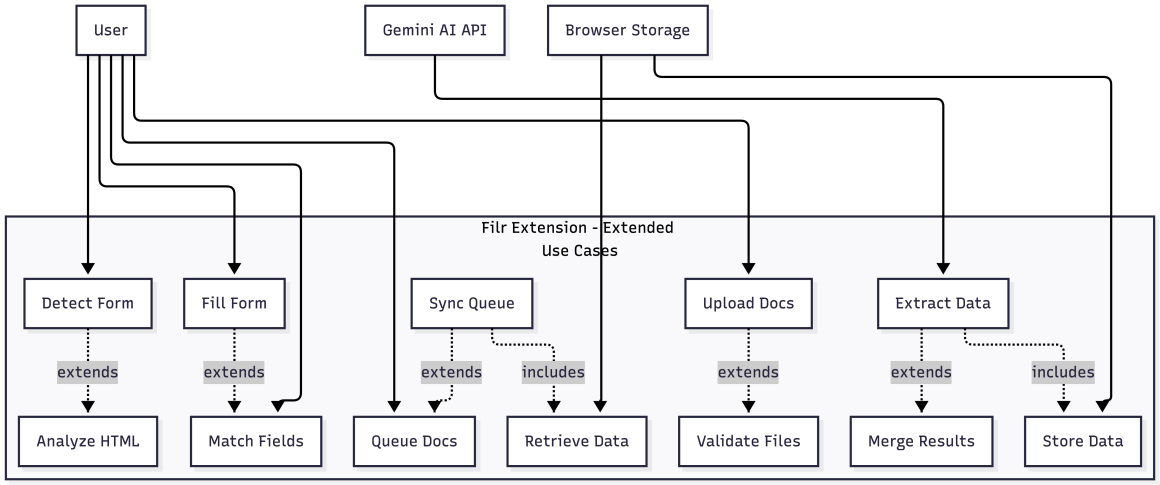

Figure 5: Extended Use Case Diagram with External Actors

Use Case Relationships

- **Extends:** Some use cases extend others, adding optional behavior (e.g., "Analyze HTML" extends "Detect Form")
- **Includes:** Some use cases include others as mandatory steps (e.g., "Extract Data" includes "Store Data")
- **External Actors:** Gemini AI API and Browser Storage are external systems that the extension interacts with

2.2.3 Use Case Descriptions

Table 1: Detailed Use Case Descriptions

Use Case	Description
Detect Form	User clicks "Detect Form" button. System extracts HTML from current page, sends to AI for analysis, and returns form structure with field metadata.
Upload Documents	User selects document files matching required document types. System validates file types and sizes, then queues or processes them based on network status.
Process Documents	System sends documents to AI API for extraction. Each document is processed through appropriate processor in the chain. Results are merged into single data object.
Auto-Fill Form	System injects script into page context, finds form fields using detected metadata, matches extracted data to fields, and fills them with proper event triggering.
Manage Queue	User views queued documents when offline. Can retry failed jobs, cancel pending jobs, or wait for automatic sync when online.
Sync Documents	When network connectivity is restored, system automatically processes all queued documents in order, with retry logic for failures.

This architecture ensures that Filr provides a robust, scalable, and maintainable solution for automated form filling, with clear separation of concerns and well-defined interaction patterns between components.

3 Design Patterns Used

The Filr extension employs multiple design patterns to achieve maintainability, scalability, and clean code architecture. This section details each pattern's implementation, rationale, and impact on the system.

3.1 Design Philosophy & Rationale

The design patterns chosen for Filr follow several key principles:

Separation of Concerns Each pattern isolates specific responsibilities, ensuring that changes to one component don't cascade throughout the system. This principle is embodied in the MVC pattern, which separates data models, user interfaces, and business logic.

Open/Closed Principle The system is open for extension but closed for modification. Patterns like Chain of Responsibility and Decorator allow adding new functionality (new document processors, new features) without modifying existing code.

Single Responsibility Each class and component has one clear purpose. Processors handle only their specific document type, observers only handle notifications, and controllers only coordinate operations.

Loose Coupling Components interact through well-defined interfaces rather than direct dependencies. The Observer pattern decouples processors from UI updates, and the Factory pattern decouples object creation from usage.

DRY (Don't Repeat Yourself) Common functionality is centralized. The Singleton pattern ensures single instances of shared resources, and the Factory pattern centralizes object creation logic.

Testability Patterns enable unit testing by allowing mock objects and isolated component testing. The Strategy pattern allows swapping implementations for testing, and the Command pattern encapsulates operations for easy testing.
These principles guide the selection and implementation of each design pattern, ensuring the codebase remains maintainable as the project evolves.

3.2 MVC (Model-View-Controller) Pattern

3.2.1 Problem Addressed

The initial implementation mixed presentation logic, business logic, and data management in a single component, leading to:

- **Tight Coupling:** UI components directly accessed storage APIs and business logic
- **Difficulty Testing:** Business logic was embedded in React components, making unit testing impossible
- **Code Duplication:** Similar logic repeated across multiple components
- **Maintenance Issues:** Changes to data structure required updates in multiple places
- **Poor Reusability:** Business logic couldn't be reused outside React components

3.2.2 Why This Pattern?

MVC provides clear separation between:

- **Models:** Data structures and business rules (e.g., DocumentModel, SettingsModel)
- **Views:** User interface components (React pages and components)
- **Controllers:** Orchestration logic connecting Models and Views

This separation enables:

- Independent development and testing of each layer
- Reuse of business logic across different interfaces
- Easier maintenance and evolution
- Clear data flow: User Action → View → Controller → Model → Controller → View Update

3.2.3 UML Diagram

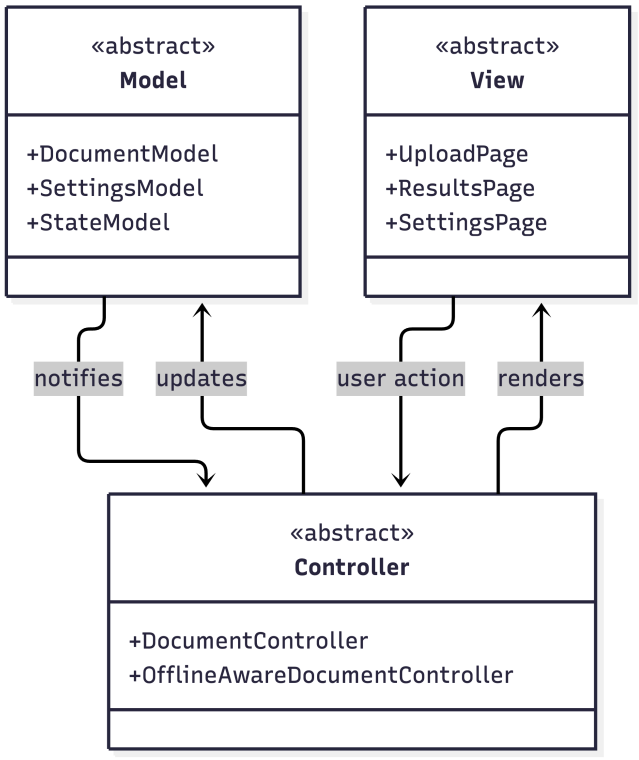

Figure 6: MVC Pattern Architecture

3.2.4 Implementation Snippet

```
// src/controllers/DocumentController.ts
export class DocumentController {
  async processDocuments(
    files: File[],
    apiKey: string,
    model: string,
    onProgress?: (event: ProgressEvent) => void
  ): Promise<ExtractedData> {
    // Controller orchestrates Model and View
    const chain = ProcessorFactory.createDefaultChain();
```

```
    chain.subscribe(onProgress);

    const result = await chain.processDocuments(files, apiKey, model);

    // Update Model
    const documentModel = new DocumentModel(result);
    return documentModel.getExtractedData();
  }
}

// src/models/DocumentModel.ts
export class DocumentModel {
  private data: ExtractedData;

  constructor(data: ExtractedData) {
    this.data = data;
  }

  getExtractedData(): ExtractedData {
    return this.data;
  }

  updateField(field: string, value: string): void {
    this.data[field] = value;
  }
}

// src/UploadPage.tsx (View)
export function UploadPage() {
  const handleProcess = async () => {
    // View calls Controller
    const result = await DocumentController.processDocuments(
      files, apiKey, model, onProgress
    );
    // View updates based on result
    setExtractedData(result);
  };
}
```

Listing 1: MVC Pattern Implementation - Controller Layer

3.2.5 Benefits Achieved

- **Testability:** Controllers can be unit tested without UI components
- **Reusability:** Business logic reused across multiple views
- **Maintainability:** Changes isolated to specific layers
- **Scalability:** Easy to add new views or modify business logic
- **Separation:** Clear boundaries between data, logic, and presentation

3.2.6 Limitations & Trade-offs

- **Complexity:** Additional abstraction layer increases initial complexity
- **Overhead:** Small applications may not benefit from full MVC separation
- **Learning Curve:** Developers must understand three-layer architecture
- **Performance:** Additional method calls between layers (negligible in practice)

3.3 Chain of Responsibility Pattern

3.3.1 Problem Addressed

The system needs to process multiple document types (birth certificates, NID cards, passports, utility bills, education certificates) where:

- Each document type requires specialized extraction logic
- Documents have different field requirements and formats
- Processing logic must be maintainable and extensible
- One document failure shouldn't crash the entire pipeline
- New document types should be addable without modifying existing code

Without this pattern, a monolithic approach would result in:

- Massive if-else chains or switch statements (300+ lines)
- Tight coupling between document types
- Difficulty adding new document types
- No separation of concerns
- Difficult testing of individual processors

3.3.2 Why This Pattern?

Chain of Responsibility allows:

- **Decoupled Processing:** Each processor is independent and handles only its document type
- **Dynamic Chain:** Processors can be added, removed, or reordered without modifying existing code
- **Fault Isolation:** Failure in one processor doesn't affect others
- **Extensibility:** New document types added by creating new processor classes
- **Testability:** Each processor can be tested independently

3.3.3 UML Diagram

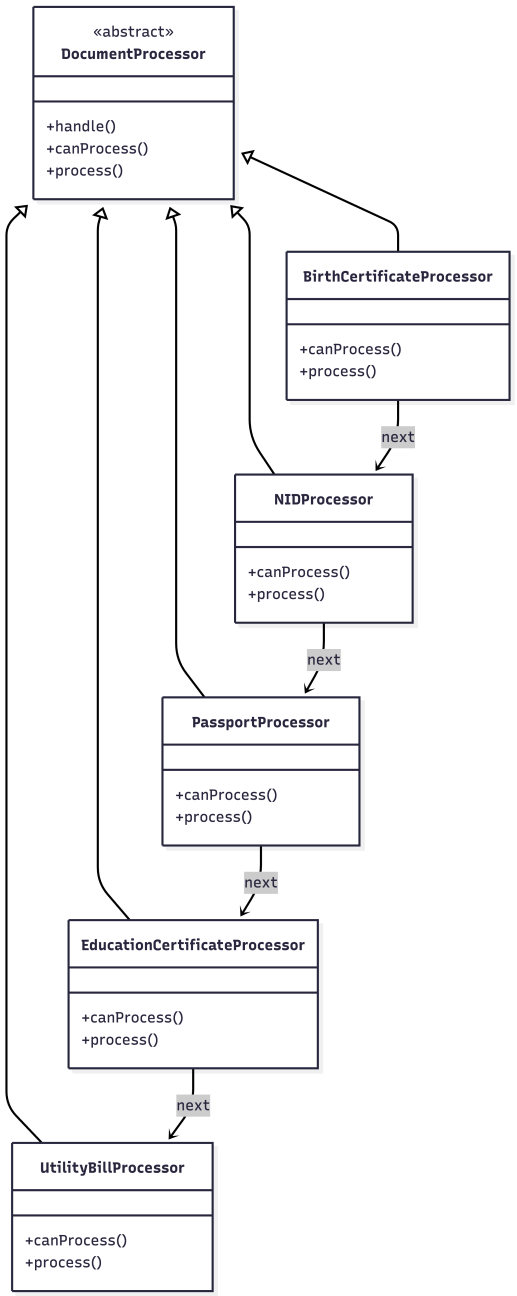

Figure 7: Chain of Responsibility Pattern Structure

3.3.4 Implementation Snippet

```
// src/lib/processors/DocumentProcessor.ts
export abstract class DocumentProcessor {
  protected next: DocumentProcessor | null = null;

  setNext(processor: DocumentProcessor): DocumentProcessor {
    this.next = processor;
    return processor;
  }

  async handle(
    file: File,
    data: ExtractedData,
    ai: GoogleGenAI,
```

```
    model: string
  ): Promise<ExtractedData> {
    if (this.canProcess(file)) {
      const result = await this.process(file, ai, model);
      data = { ...data, ...result };
    }

    // Pass to next processor in chain
    if (this.next) {
      return await this.next.handle(file, data, ai, model);
    }

    return data;
  }

  abstract canProcess(file: File): boolean;
  abstract process(
    file: File,
    ai: GoogleGenAI,
    model: string
  ): Promise<Partial<ExtractedData>>;
}

// src/lib/processors/BirthCertificateProcessor.ts
export class BirthCertificateProcessor extends DocumentProcessor {
  canProcess(file: File): boolean {
    return file.name.toLowerCase().includes('birth');
  }

  async process(file: File, ai: GoogleGenAI, model: string) {
    // Specific birth certificate extraction logic
    return await extractBirthCertificateData(file, ai, model);
  }
}

// Chain construction
const chain = new BirthCertificateProcessor()
  .setNext(new NIDProcessor())
  .setNext(new PassportProcessor())
  .setNext(new EducationCertificateProcessor())
  .setNext(new UtilityBillProcessor());
```

Listing 2: Chain of Responsibility Implementation

3.3.5 Benefits Achieved

- **Extensibility:** New processors added without modifying existing code
- **Fault Tolerance:** One processor failure doesn't break the chain
- **Maintainability:** Each processor is self-contained and testable
- **Flexibility:** Chain can be dynamically constructed based on file types
- **Single Responsibility:** Each processor handles one document type

3.3.6 Limitations & Trade-offs

- **Performance:** Files may pass through multiple processors before matching
- **Debugging:** Chain traversal can be harder to debug than direct calls
- **Guarantee:** No guarantee that a file will be processed (if no processor matches)
- **Order Dependency:** Chain order matters for efficiency

3.4 Observer Pattern

3.4.1 Problem Addressed

The system needs to notify multiple components about processing progress without:

- Tight coupling between processors and UI components
- Hard-coding specific notification targets
- Difficulty adding new notification mechanisms
- Polling for status updates (inefficient)

Without this pattern, processors would need direct references to UI components, creating tight coupling and making the system difficult to extend.

3.4.2 Why This Pattern?

Observer pattern provides:

- **Loose Coupling:** Processors don't know about specific observers
- **Dynamic Subscription:** Observers can subscribe/unsubscribe at runtime
- **Multiple Observers:** One subject can notify many observers
- **Extensibility:** New observers added without modifying subjects
- **Event-Driven:** Real-time updates without polling

3.4.3 UML Diagram

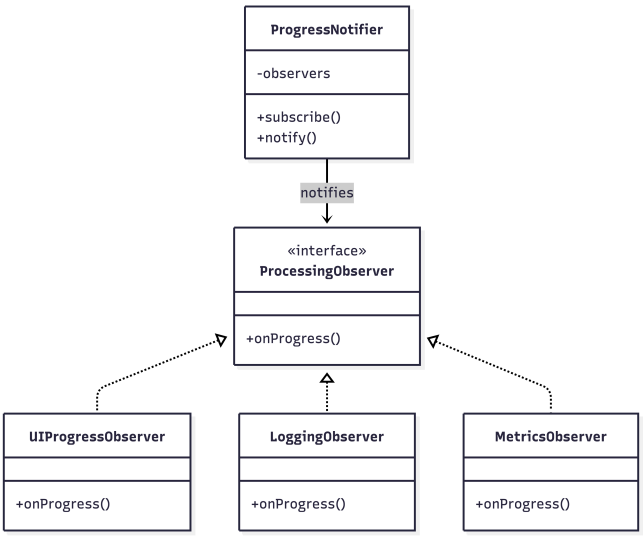

Figure 8: Observer Pattern Architecture

3.4.4 Implementation Snippet

```
// src/lib/observers/ProcessingObserver.ts
export interface ProcessingObserver {
  onProgress(event: ProgressEvent): void;
}

export interface ProgressEvent {
```

```
  fileIndex: number;
  totalFiles: number;
  fileName: string;
  status: 'processing' | 'completed' | 'error';
  progress: number;
}

export class ProgressNotifier {
  private observers: ProcessingObserver[] = [];

  subscribe(observer: ProcessingObserver): void {
    this.observers.push(observer);
  }

  unsubscribe(observer: ProcessingObserver): void {
    this.observers = this.observers.filter(o => o !== observer);
  }

  notify(event: ProgressEvent): void {
    this.observers.forEach(observer => observer.onProgress(event));
  }
}

// Usage in ProcessorChain
export class ProcessorChain {
  private notifier = new ProgressNotifier();

  subscribe(observer: ProcessingObserver): void {
    this.notifier.subscribe(observer);
  }

  async processDocuments(files: File[]) {
    for (let i = 0; i < files.length; i++) {
      this.notifier.notify({
        fileIndex: i,
        totalFiles: files.length,
        fileName: files[i].name,
        status: 'processing',
        progress: (i / files.length) * 100
      });
      // Process file...
    }
  }
}

// UI Component subscribes
const chain = new ProcessorChain();
chain.subscribe({
  onProgress: (event) => {
    setProgress(event.progress);
    updateUI(event);
  }
});
```

Listing 3: Observer Pattern Implementation

3.4.5 Benefits Achieved

- **Decoupling:** Processors independent of UI implementation
- **Flexibility:** Multiple observers can react to same events
- **Real-time Updates:** UI updates immediately without polling
- **Extensibility:** New observers (logging, analytics) added easily

- **Testability:** Mock observers for testing

3.4.6 Limitations & Trade-offs

- **Memory:** Observers held in memory until unsubscribed
- **Order:** No guarantee of observer notification order
- **Debugging:** Event flow can be harder to trace
- **Performance:** Many observers may impact performance

3.5 Factory Pattern

3.5.1 Problem Addressed

Creating processor chains requires:

- Complex chain construction logic scattered across codebase
- Knowledge of all processor types in multiple places
- Difficulty optimizing chain based on file types
- Tight coupling between chain users and processor implementations

3.5.2 Why This Pattern?

Factory pattern centralizes object creation:

- **Encapsulation:** Complex creation logic hidden in factory
- **Flexibility:** Different chain configurations created easily
- **Optimization:** Factory can create optimized chains based on input
- **Maintainability:** Changes to chain structure isolated to factory
- **Testability:** Factory can be mocked for testing

3.5.3 UML Diagram

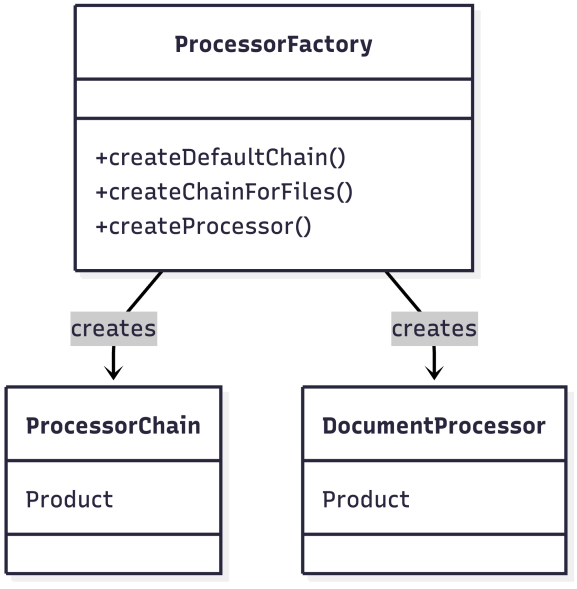

Figure 9: Factory Pattern Architecture

3.5.4 Implementation Snippet

```
// src/lib/factories/ProcessorFactory.ts
export class ProcessorFactory {
  static createDefaultChain(): ProcessorChain {
    const chain = new ProcessorChain();
    chain.addProcessor(new BirthCertificateProcessor());
    chain.addProcessor(new NIDProcessor());
    chain.addProcessor(new PassportProcessor());
    chain.addProcessor(new EducationCertificateProcessor());
    chain.addProcessor(new UtilityBillProcessor());
    return chain;
  }

  static createChainForFiles(files: File[]): ProcessorChain {
    const chain = new ProcessorChain();
    const fileTypes = new Set(files.map(f => this.detectFileType(f)));

    // Only add processors for detected file types
    if (fileTypes.has('birth')) {
      chain.addProcessor(new BirthCertificateProcessor());
    }
    if (fileTypes.has('nid')) {
      chain.addProcessor(new NIDProcessor());
    }
    // ... optimize based on actual files

    return chain;
  }

  static createProcessor(type: string): DocumentProcessor {
    switch (type) {
      case 'birth': return new BirthCertificateProcessor();
      case 'nid': return new NIDProcessor();
      case 'passport': return new PassportProcessor();
      default: throw new Error(`Unknown processor type: ${type}`);
    }
  }

  private static detectFileType(file: File): string {
    const name = file.name.toLowerCase();
    if (name.includes('birth')) return 'birth';
    if (name.includes('nid')) return 'nid';
    // ... detection logic
    return 'unknown';
  }
}
```

Listing 4: Factory Pattern Implementation

3.5.5 Benefits Achieved

- **Centralization:** All creation logic in one place
- **Optimization:** Factory can create optimized chains
- **Flexibility:** Multiple creation methods for different scenarios
- **Maintainability:** Changes to creation logic isolated
- **Testability:** Factory can be mocked or replaced

3.5.6 Limitations & Trade-offs

- **Complexity:** Factory class can become complex with many product types
- **God Object:** Factory may become too large if not carefully designed
- **Static Methods:** Harder to mock in some testing frameworks

3.6 Singleton Pattern

3.6.1 Problem Addressed

Critical services need:

- Single instance across entire application
- Global access point
- Resource sharing (API clients, settings)
- Consistent state management

Multiple instances would cause:

- Resource waste (multiple API clients)
- Inconsistent state (different settings instances)
- Configuration conflicts

3.6.2 Why This Pattern?

Singleton ensures:

- **Single Instance:** Only one instance exists
- **Global Access:** Accessible from anywhere
- **Lazy Initialization:** Created only when needed
- **Resource Efficiency:** Shared resources (API connections)
- **State Consistency:** Single source of truth

3.6.3 UML Diagram

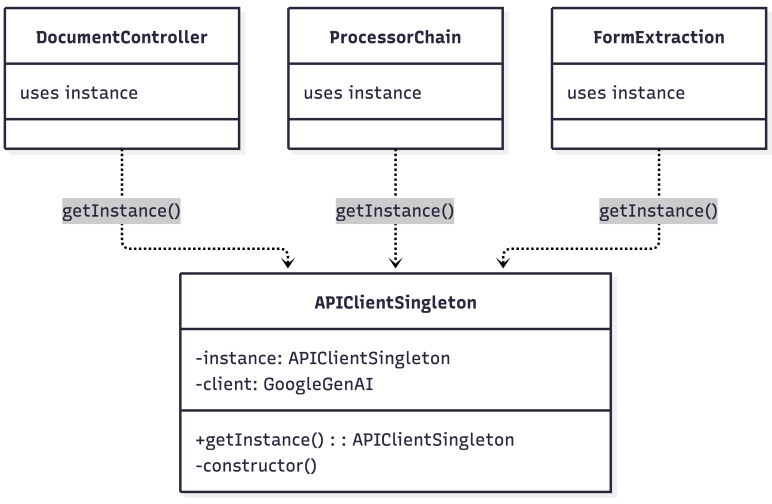

Figure 10: Singleton Pattern Architecture

3.6.4 Implementation Snippet

```typescript
// src/lib/singleton/APIClientSingleton.ts
export class APIClientSingleton {
  private static instance: APIClientSingleton | null = null;
  private client: GoogleGenAI | null = null;

  private constructor() {
    // Private constructor prevents direct instantiation
  }

  static getInstance(): APIClientSingleton {
    if (!APIClientSingleton.instance) {
      APIClientSingleton.instance = new APIClientSingleton();
    }
    return APIClientSingleton.instance;
  }

  initializeClient(apiKey: string): void {
    if (!this.client) {
      this.client = new GoogleGenAI({ apiKey });
    }
  }

  getClient(): GoogleGenAI {
    if (!this.client) {
      throw new Error('Client not initialized');
    }
    return this.client;
  }
}

// Usage throughout application
const apiClient = APIClientSingleton.getInstance();
apiClient.initializeClient(apiKey);
const client = apiClient.getClient();
```

Listing 5: Singleton Pattern Implementation

3.6.5 Benefits Achieved

- **Resource Efficiency:** Single API client instance
- **State Consistency:** Shared state across application
- **Global Access:** Accessible from anywhere
- **Lazy Loading:** Created only when needed
- **Configuration:** Single configuration point

3.6.6 Limitations & Trade-offs

- **Testing:** Harder to mock or replace in tests
- **Global State:** Can lead to hidden dependencies
- **Thread Safety:** Not applicable in JavaScript (single-threaded)
- **Anti-pattern Concerns:** Some consider it an anti-pattern due to global state

3.7 State Pattern

3.7.1 Problem Addressed

Application navigation and state management requires:

- Valid state transitions (prevent invalid navigation)
- State-specific behavior (different actions per state)
- State context management (data associated with states)
- Clear state machine definition

Without this pattern, state management becomes:

- Scattered if-else statements throughout code
- No validation of state transitions
- Difficult to add new states
- Hard to reason about application flow

3.7.2 Why This Pattern?

State pattern provides:

- **Encapsulation:** Each state encapsulates its behavior
- **Transition Control:** Valid transitions defined per state
- **Extensibility:** New states added without modifying existing ones
- **Clarity:** Clear state machine representation
- **Testability:** Each state can be tested independently

3.7.3 UML Diagram

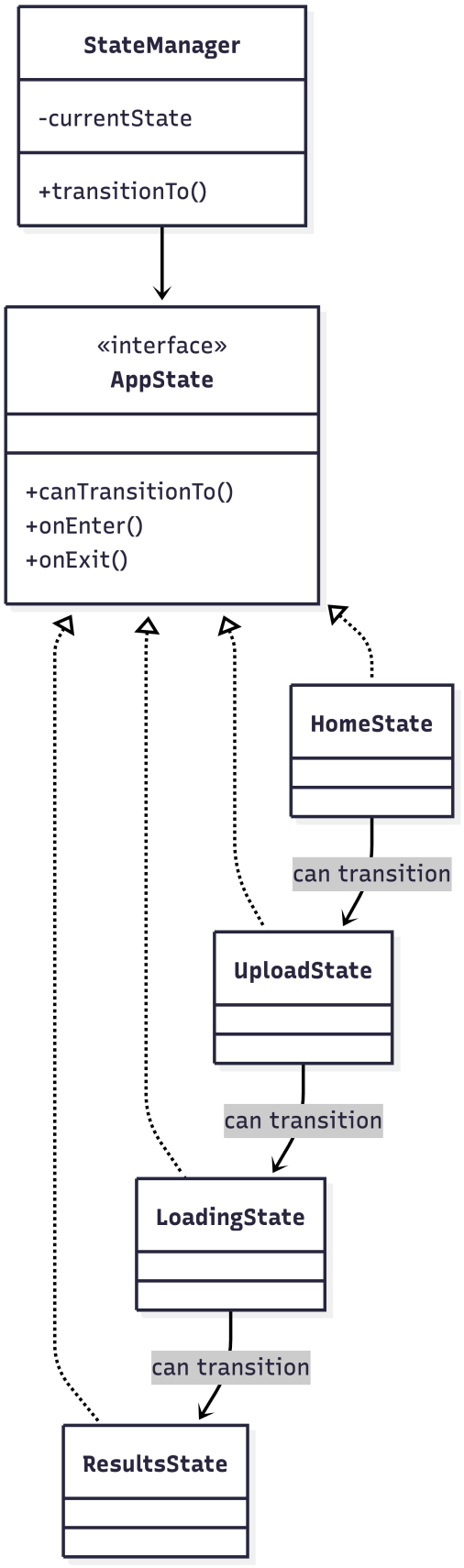

Figure 11: State Pattern Architecture

3.7.4 Implementation Snippet

```
// src/lib/state/AppState.ts
export interface AppState {
  readonly name: PageType;
  canTransitionTo(targetState: PageType): boolean;
  onEnter(context: StateContext): void;
  onExit(context: StateContext): void;
}

export class HomeState implements AppState {
  readonly name: PageType = 'home';

  canTransitionTo(targetState: PageType): boolean {
    return ['upload', 'settings', 'formdetection'].includes(targetState);
  }

  onEnter(context: StateContext): void {
    context.currentPage = 'home';
  }

  onExit(context: StateContext): void {
    // Cleanup if needed
  }
}

export class StateManager {
  private currentState: AppState;
  private context: StateContext;

  constructor(initialState: AppState, context: StateContext) {
    this.currentState = initialState;
    this.context = context;
    this.currentState.onEnter(context);
  }

  transitionTo(targetState: PageType): boolean {
    if (!this.currentState.canTransitionTo(targetState)) {
      console.error(`Invalid transition from ${this.currentState.name} to ${targetState}`);
      return false;
    }

    this.currentState.onExit(this.context);
    this.currentState = this.getStateForPage(targetState);
    this.currentState.onEnter(this.context);
    return true;
  }
}
```

Listing 6: State Pattern Implementation

3.7.5 Benefits Achieved

- **Type Safety:** Invalid transitions prevented at compile time
- **Clarity:** Clear state machine representation
- **Maintainability:** State logic encapsulated per state
- **Extensibility:** New states added easily
- **Testability:** Each state tested independently

3.7.6 Limitations & Trade-offs

- **Complexity:** More classes for simple state machines
- **Overhead:** Additional abstraction layer
- **State Explosion:** Many states can lead to many classes

3.8 Decorator Pattern

3.8.1 Problem Addressed

Need to add functionality to processors (logging, metrics, rate limiting) without:

- Modifying existing processor code
- Creating subclasses for every combination
- Tight coupling between processors and cross-cutting concerns

3.8.2 Why This Pattern?

Decorator pattern allows:

- **Dynamic Composition:** Functionality added at runtime
- **Flexibility:** Multiple decorators can be combined
- **Open/Closed:** Open for extension, closed for modification
- **Single Responsibility:** Each decorator adds one concern
- **Composition over Inheritance:** Avoids class explosion

3.8.3 UML Diagram

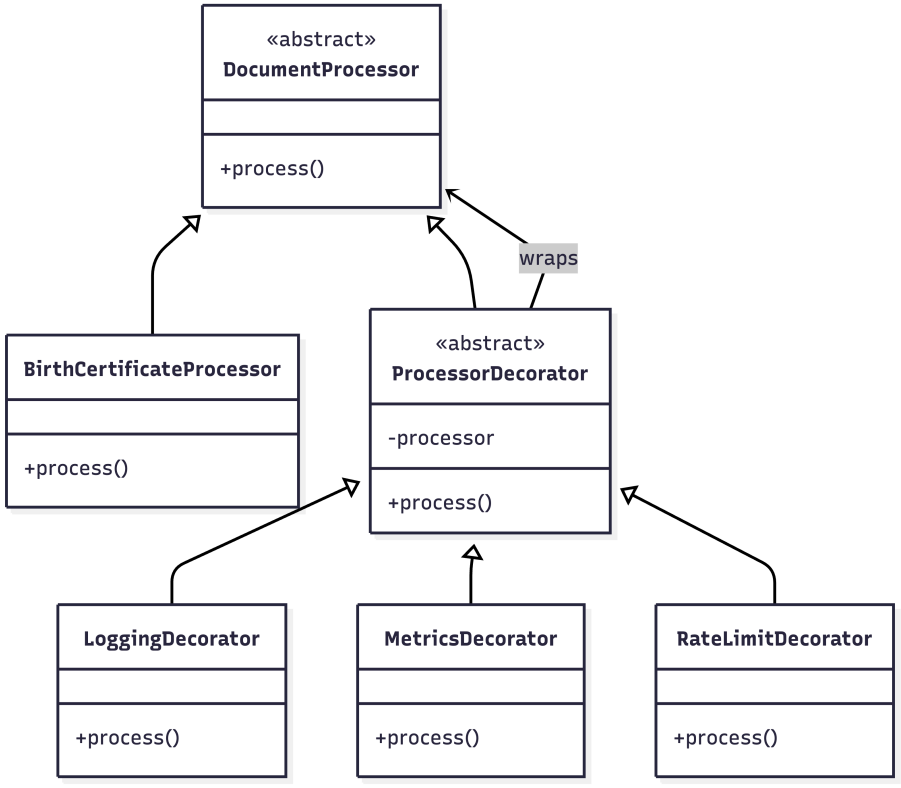

Figure 12: Decorator Pattern Architecture

3.8.4 Implementation Snippet

```
// src/lib/decorators/ProcessorDecorator.ts
export abstract class ProcessorDecorator extends DocumentProcessor {
  protected processor: DocumentProcessor;

  constructor(processor: DocumentProcessor) {
    super();
    this.processor = processor;
  }

  async process(file: File, ai: GoogleGenAI, model: string) {
    return await this.processor.process(file, ai, model);
  }
}

export class LoggingProcessorDecorator extends ProcessorDecorator {
  async process(file: File, ai: GoogleGenAI, model: string) {
    console.log(`[LOG] Processing: ${file.name}`);
    const startTime = Date.now();

    const result = await super.process(file, ai, model);

    const duration = Date.now() - startTime;
    console.log(`[LOG] Completed in ${duration}ms`);

    return result;
  }
}

export class MetricsProcessorDecorator extends ProcessorDecorator {
  async process(file: File, ai: GoogleGenAI, model: string) {
    const result = await super.process(file, ai, model);
    MetricsCollector.recordProcessing(file.name, result);
    return result;
  }
}

// Usage: Compose decorators
const processor = new BirthCertificateProcessor();
const loggedProcessor = new LoggingProcessorDecorator(processor);
const metricsProcessor = new MetricsProcessorDecorator(loggedProcessor);
```

Listing 7: Decorator Pattern Implementation

3.8.5 Benefits Achieved

- **Flexibility:** Functionality added/removed dynamically
- **Composition:** Multiple decorators can be combined
- **Separation:** Cross-cutting concerns separated from business logic
- **Reusability:** Decorators reusable across processors
- **Testability:** Each decorator tested independently

3.8.6 Limitations & Trade-offs

- **Complexity:** Many decorators can be hard to understand
- **Order Dependency:** Decorator order matters
- **Debugging:** Stack of decorators can be harder to debug

- **Performance:** Additional method calls (usually negligible)

3.9 Strategy Pattern

3.9.1 Problem Addressed

The system needs to support different extraction strategies:

- Static extraction (predefined schemas)
- Dynamic extraction (form-based schemas)
- Future extraction methods

Without this pattern, switching strategies would require:

- Conditional logic scattered throughout code
- Difficulty adding new strategies
- Tight coupling to specific strategies

3.9.2 Why This Pattern?

Strategy pattern enables:

- **Interchangeability:** Strategies swapped at runtime
- **Encapsulation:** Each strategy encapsulates its algorithm
- **Extensibility:** New strategies added without modifying clients
- **Testability:** Each strategy tested independently
- **Separation:** Algorithm selection separated from usage

3.9.3 UML Diagram

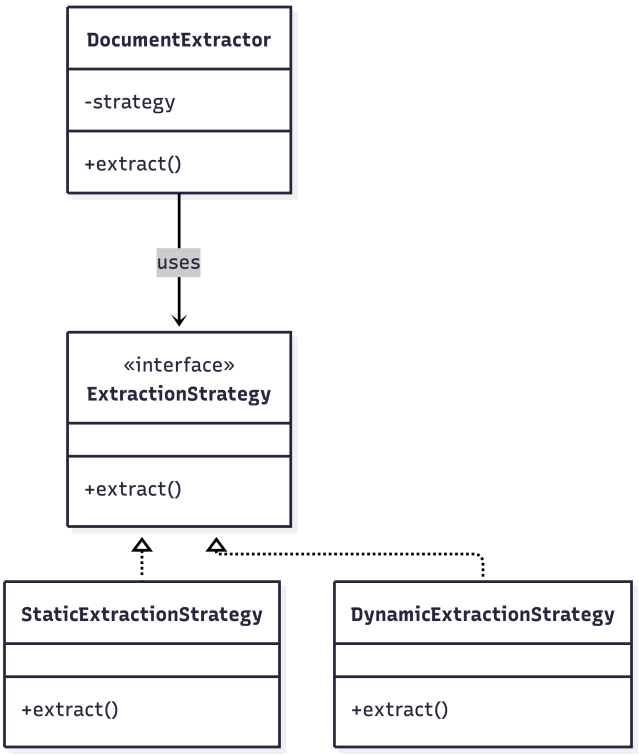

Figure 13: Strategy Pattern Architecture

3.9.4 Implementation Snippet

```
// src/lib/strategies/ExtractionStrategy.ts
export interface ExtractionStrategy {
  extract(
    file: File,
    ai: GoogleGenAI,
    model: string,
    schema: any
  ): Promise<ExtractedData>;
}

export class StaticExtractionStrategy implements ExtractionStrategy {
  async extract(
    file: File,
    ai: GoogleGenAI,
    model: string,
    schema: any
  ) {
    // Use predefined schema for known document types
    return await this.extractWithStaticSchema(file, ai, model);
  }
}

export class DynamicExtractionStrategy implements ExtractionStrategy {
  async extract(
    file: File,
    ai: GoogleGenAI,
    model: string,
    schema: any
  ) {
    // Build schema dynamically based on form fields
    return await this.extractWithDynamicSchema(file, ai, model, schema);
  }
}

export class DocumentExtractor {
  private strategy: ExtractionStrategy;

  constructor(strategy: ExtractionStrategy) {
    this.strategy = strategy;
  }

  setStrategy(strategy: ExtractionStrategy): void {
    this.strategy = strategy;
  }

  async extract(file: File, ai: GoogleGenAI, model: string, schema: any) {
    return await this.strategy.extract(file, ai, model, schema);
  }
}

// Usage
const extractor = new DocumentExtractor(new StaticExtractionStrategy());
// Switch to dynamic strategy
extractor.setStrategy(new DynamicExtractionStrategy());
```

Listing 8: Strategy Pattern Implementation

3.9.5 Benefits Achieved

- **Flexibility:** Strategies swapped at runtime
- **Extensibility:** New strategies added easily

- **Testability:** Each strategy tested independently
- **Maintainability:** Strategy logic isolated
- **Reusability:** Strategies reusable in different contexts

3.9.6 Limitations & Trade-offs

- **Overhead:** Additional abstraction layer
- **Client Awareness:** Clients must know about different strategies
- **Strategy Selection:** Need logic to choose appropriate strategy

3.10 Summary of Design Patterns

The Filr extension successfully implements 12 design patterns, each addressing specific architectural concerns:

Table 2: Design Patterns Summary

Pattern	Category	Primary Benefit
MVC	Architectural	Separation of concerns
Chain of Responsibility	Behavioral	Extensible processing pipeline
Observer	Behavioral	Decoupled event notifications
Factory	Creational	Centralized object creation
Singleton	Creational	Single instance management
State	Behavioral	Controlled state transitions
Decorator	Structural	Dynamic functionality addition
Strategy	Behavioral	Algorithm interchangeability
Command	Behavioral	Operation encapsulation
Prototype	Creational	Object cloning
Mediator	Behavioral	Component communication
Proxy	Structural	Request interception

These patterns work together to create a maintainable, extensible, and testable codebase that follows software engineering best practices.

4 System Implementation

This section provides a comprehensive overview of the Filr extension's implementation details, including user interface screens, code organization, file structure, and data storage mechanisms.

4.1 UI Screens

The Filr extension provides a modern, intuitive user interface through Chrome's side panel. The application consists of multiple screens, each serving a specific purpose in the document processing workflow.

4.1.1 Home Screen

The home screen (App.tsx) serves as the main entry point, providing navigation to all major features:

- **Application Logo:** Filr logo with gradient background and decorative ring
- **Detect Form Button:** Initiates form detection on the current web page
- **Update Info Button:** Opens traditional form entry page for manual data input
- **Settings Button:** Accesses application configuration (positioned in top-right corner)
- **Network Status Indicator:** Shows current online/offline status
- **Toast Container:** Displays notifications for various operations
- **Application Description:** "AI-Powered Form Assistant" tagline

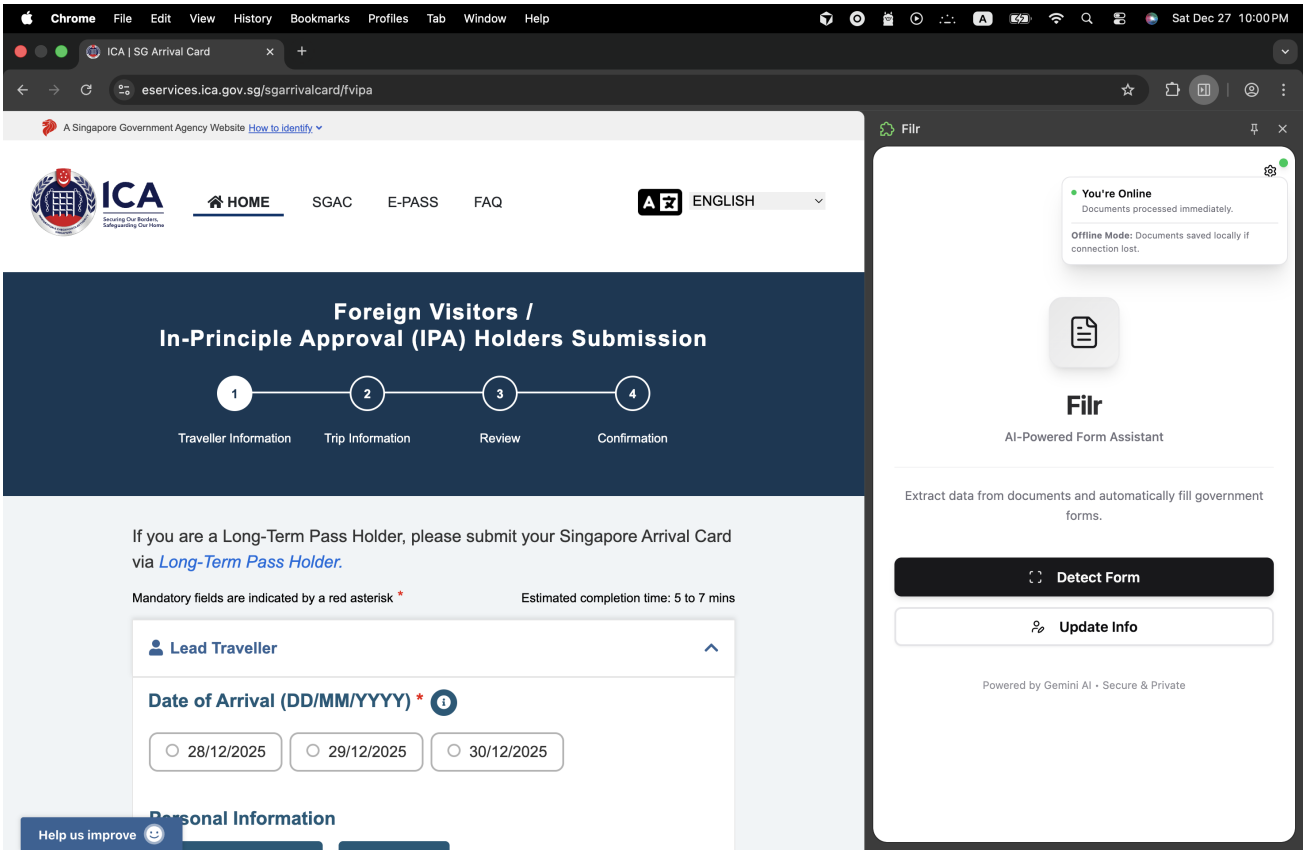

Figure 14: Home page

4.1.2 Form Detection Screen

This screen (FormDetectionPage.tsx) handles automatic form field detection with step-by-step progress:

- **Detection Progress Steps:** Visual step indicators showing current detection phase
- **Cache Management:** Resume dialog for previously detected forms with cache information
- **Form Fields Detection:** Automatic extraction of form elements from web pages using accessibility tree
- **Required Documents Analysis:** AI-powered identification of needed documents based on form structure
- **Continue Button:** Proceeds to document upload after successful detection
- **Error Handling:** Displays detection failures with clear error messages and retry options
- **Back Navigation:** Return to home screen option

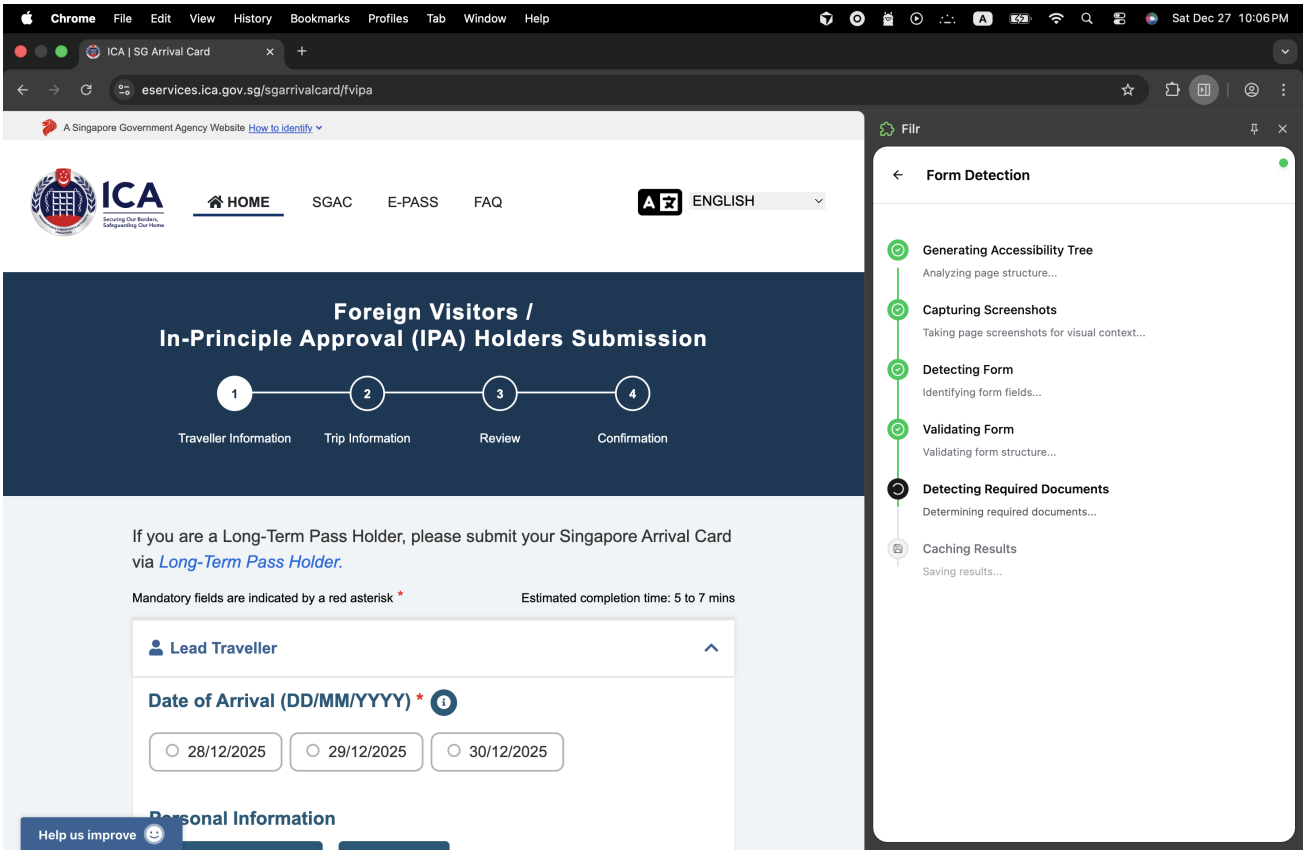

Figure 15: Form detection page

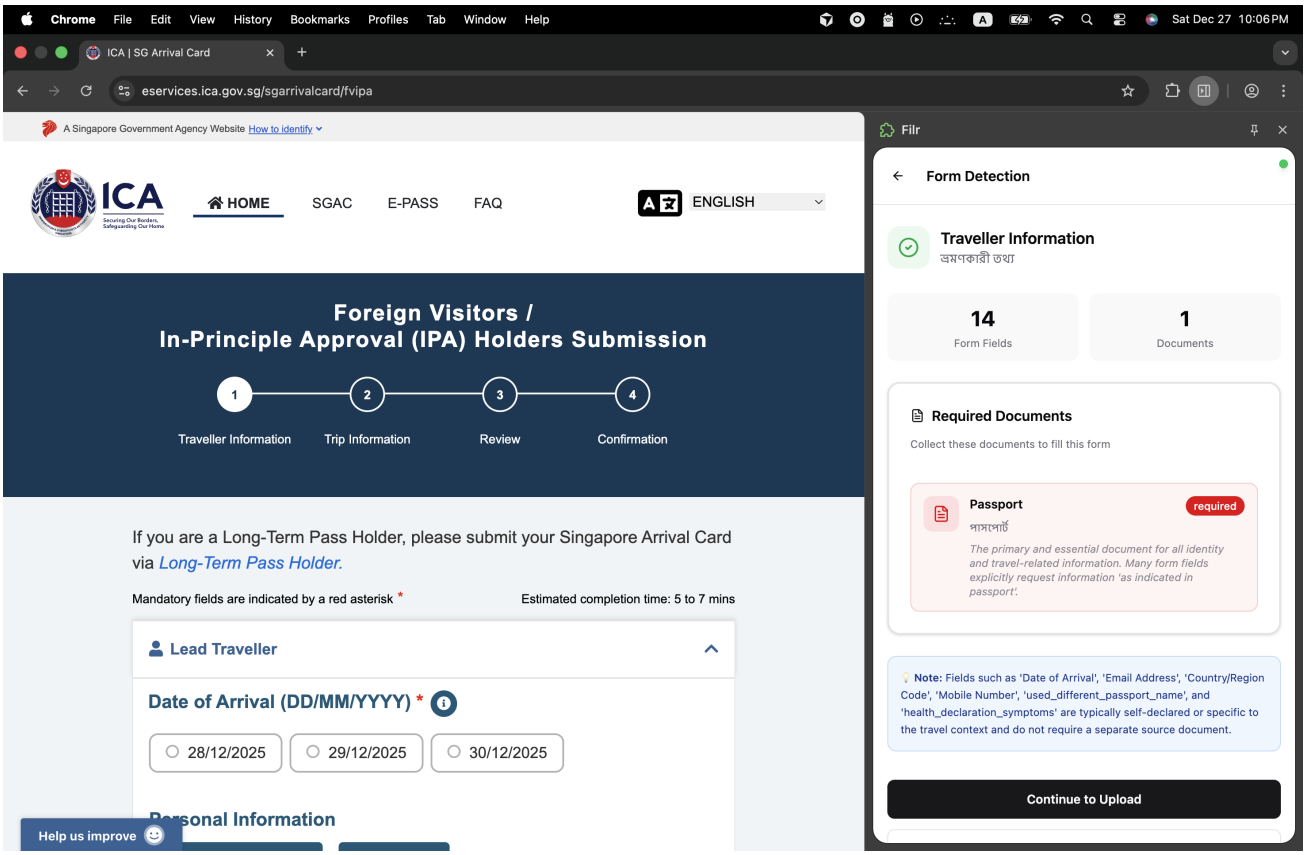

Figure 16: List of required documents

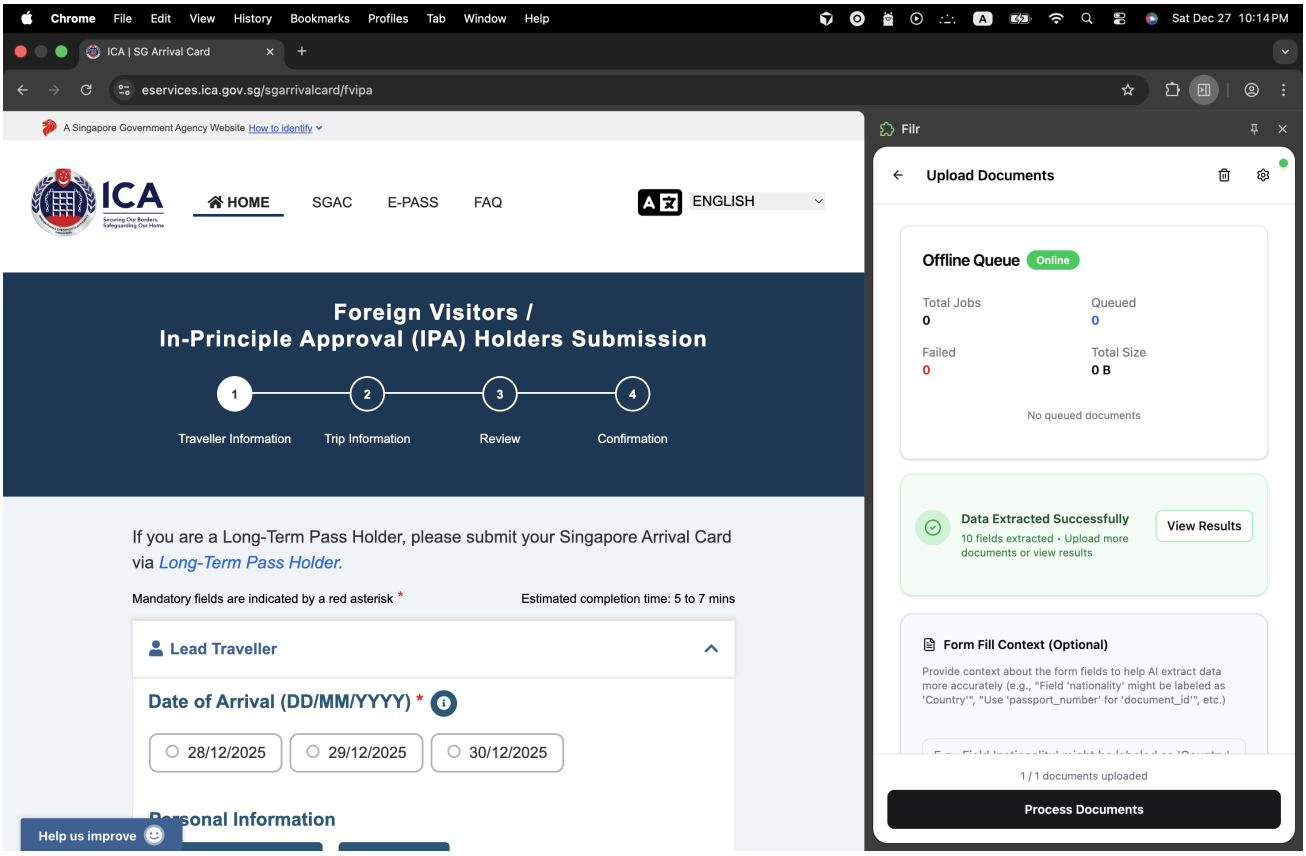

Figure 17: Cache data reusal dialog

4.1.3 Document Upload Screen

The upload interface (UploadPage.tsx) allows users to select and upload documents:

- **Document Categories:** Pre-configured document types (Birth Certificate, Utility Bill, Education Certificate, NID Card, Passport, Other Documents)
- **File Upload Areas:** Individual upload zones for each document type with drag-and-drop support
- **Document Requirements:** Visual indicators showing required vs optional documents
- **File Validation:** Type and size validation with detailed error messages
- **Process Documents Button:** Initiates AI-powered document processing workflow
- **Clear All Button:** Removes all uploaded files at once
- **Queue Status Display:** Shows queued documents count when offline
- **Network Status Banner:** Real-time connectivity alerts and notifications
- **Progress Integration:** Seamless transition to processing screen with progress callbacks
- **View Results Button:** Direct access to previously processed results

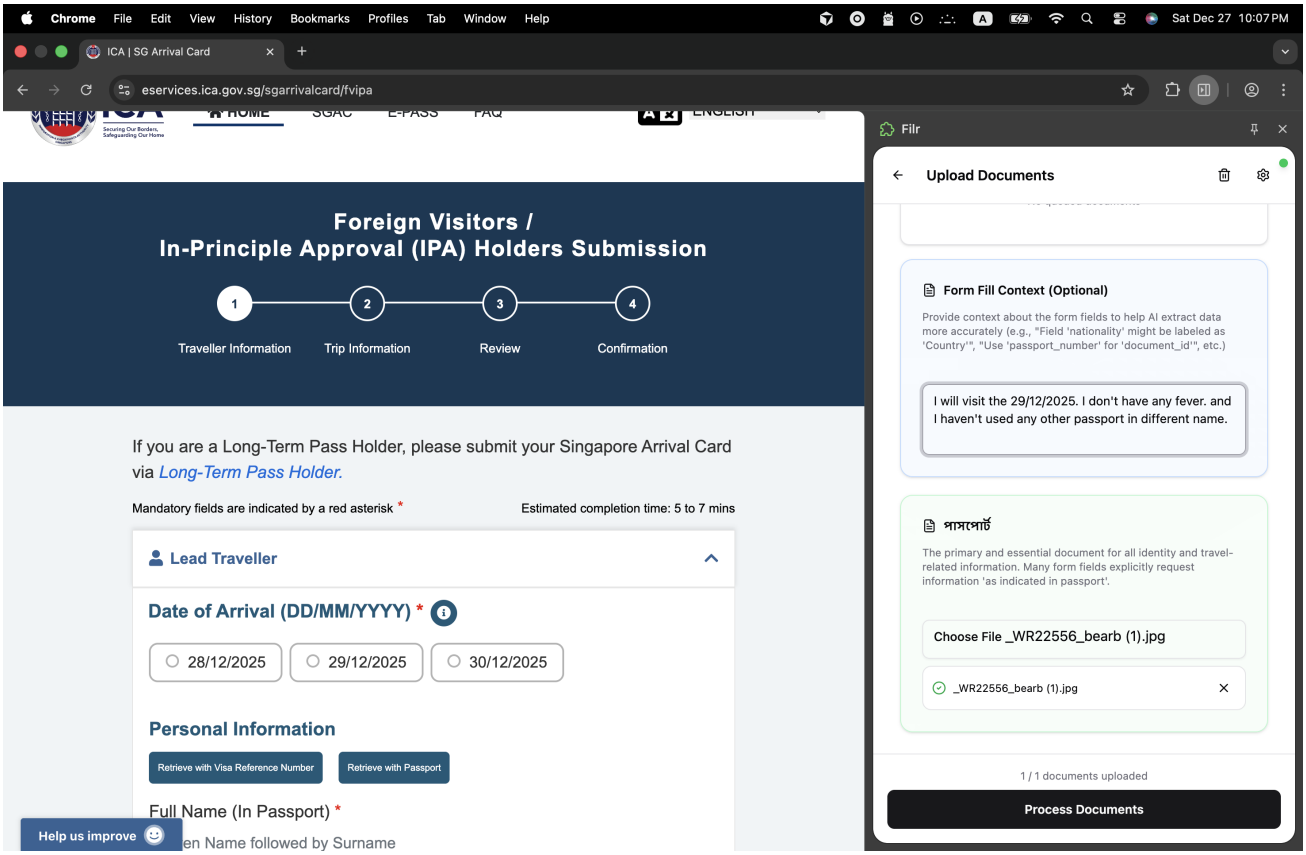

Figure 18: File upload page with custom context input

4.1.4 Processing Screen

Real-time progress display (ProcessingScreen.tsx) during document processing:

- **File Progress Cards:** Individual status cards for each document being processed
- **Status Icons:** Visual indicators (uploading, processing, completed, error states)

- **Progress Counter:** Displays "File X of Y" completion status
- **Real-time Updates:** Live status messages from processing pipeline using Observer pattern
- **Error Display:** Detailed error messages for failed processing with specific failure reasons
- **Processing States:** Clear visual distinction between pending, in-progress, and completed files

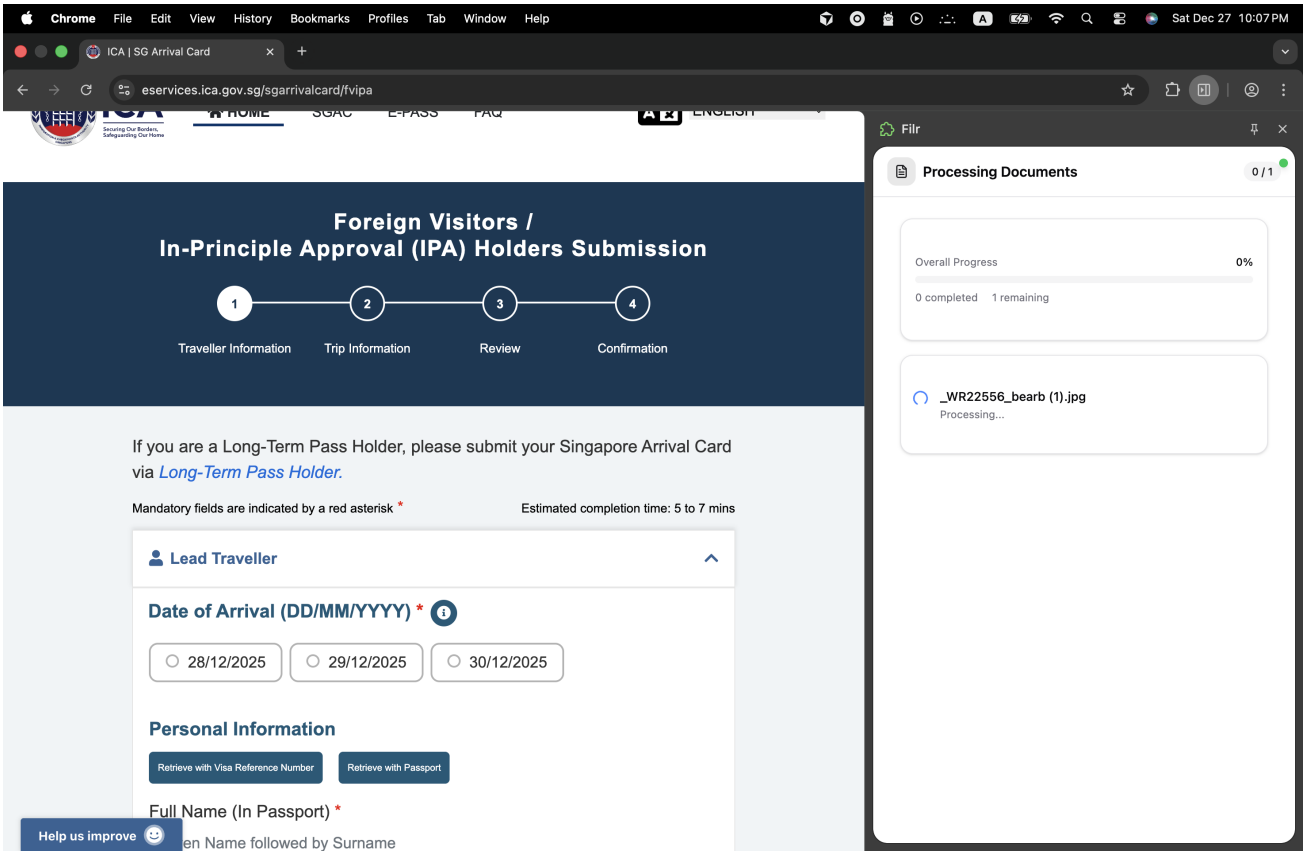

Figure 19: Document processing

4.1.5 Results Screen

Displays extracted data (ResultsPage.tsx) in an organized, editable format:

- **Dynamic Data Display:** Adapts layout based on detected form structure (dynamic vs static data)
- **Editable Fields:** All extracted values can be manually corrected with real-time updates
- **Field Categories:** Organized by data type (personal information, address, education, family details)
- **Auto-Fill Form Button:** Triggers automatic form filling on detected web pages
- **Auto-Fill Results:** Detailed feedback showing successful/failed field mappings with statistics
- **Back Navigation:** Context-aware return to previous screen (upload or traditional form)
- **Confirm Button:** Saves data and returns to form entry (available when accessed from traditional form)
- **Toast Notifications:** Success and error feedback for all operations

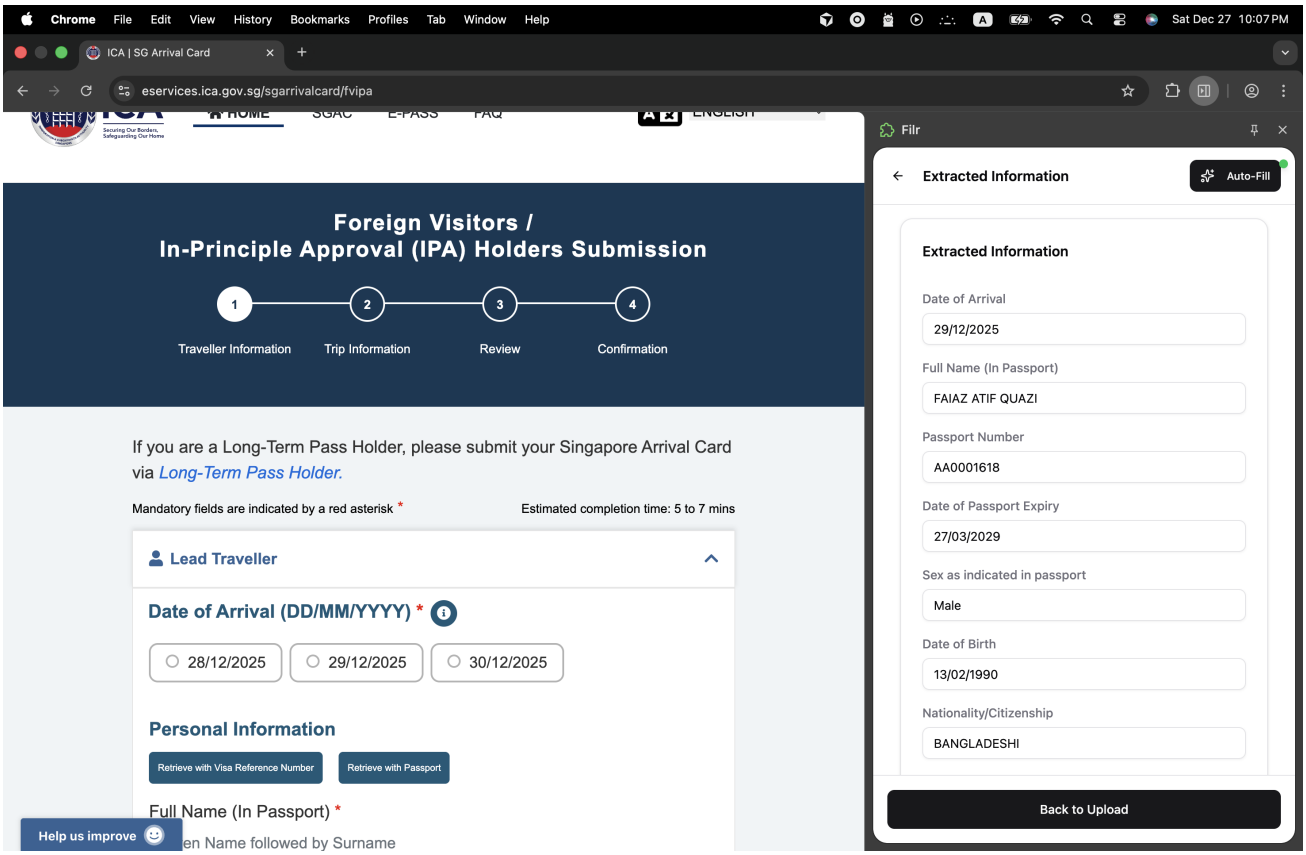

Figure 20: Extracted information page

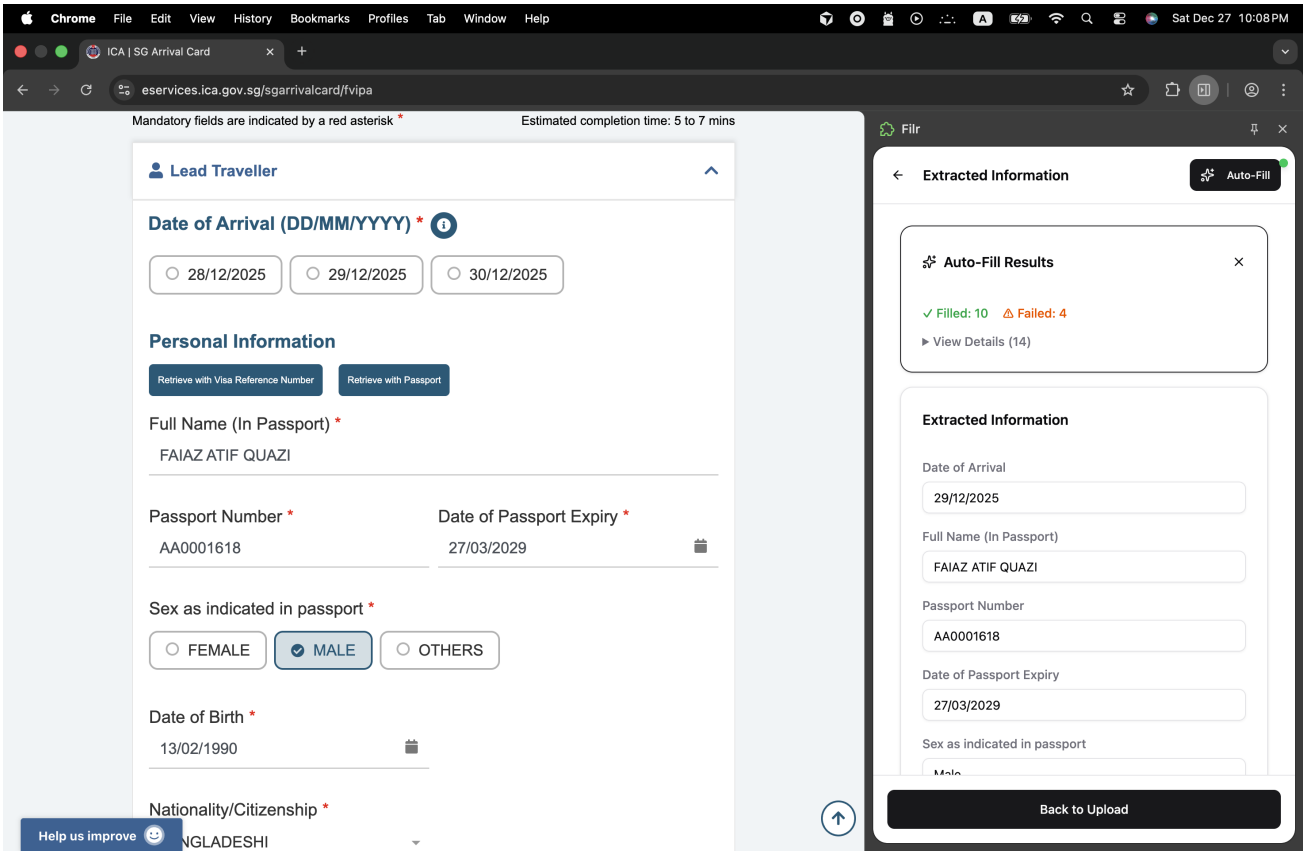

Figure 21: Form fillup success error state

4.1.6 Settings Screen

Application configuration and preferences (SettingsPage.tsx):

- **API Configuration Tab:** Google Gemini API key management with secure input and password visibility toggle
- **Model Selection:** Choose AI models (gemini-1.5-pro, gemini-1.5-flash) with performance indicators
- **Processing Settings:** Batch processing configuration and timeout settings
- **Privacy & Security:** Offline encryption password management with strength indicators
- **Cache Management:** View and clear form detection cache with detailed cache information
- **Test Connection:** Validate API key and model accessibility with real-time testing
- **Import/Export:** Configuration backup and restore functionality
- **Debug Mode:** Development and troubleshooting options

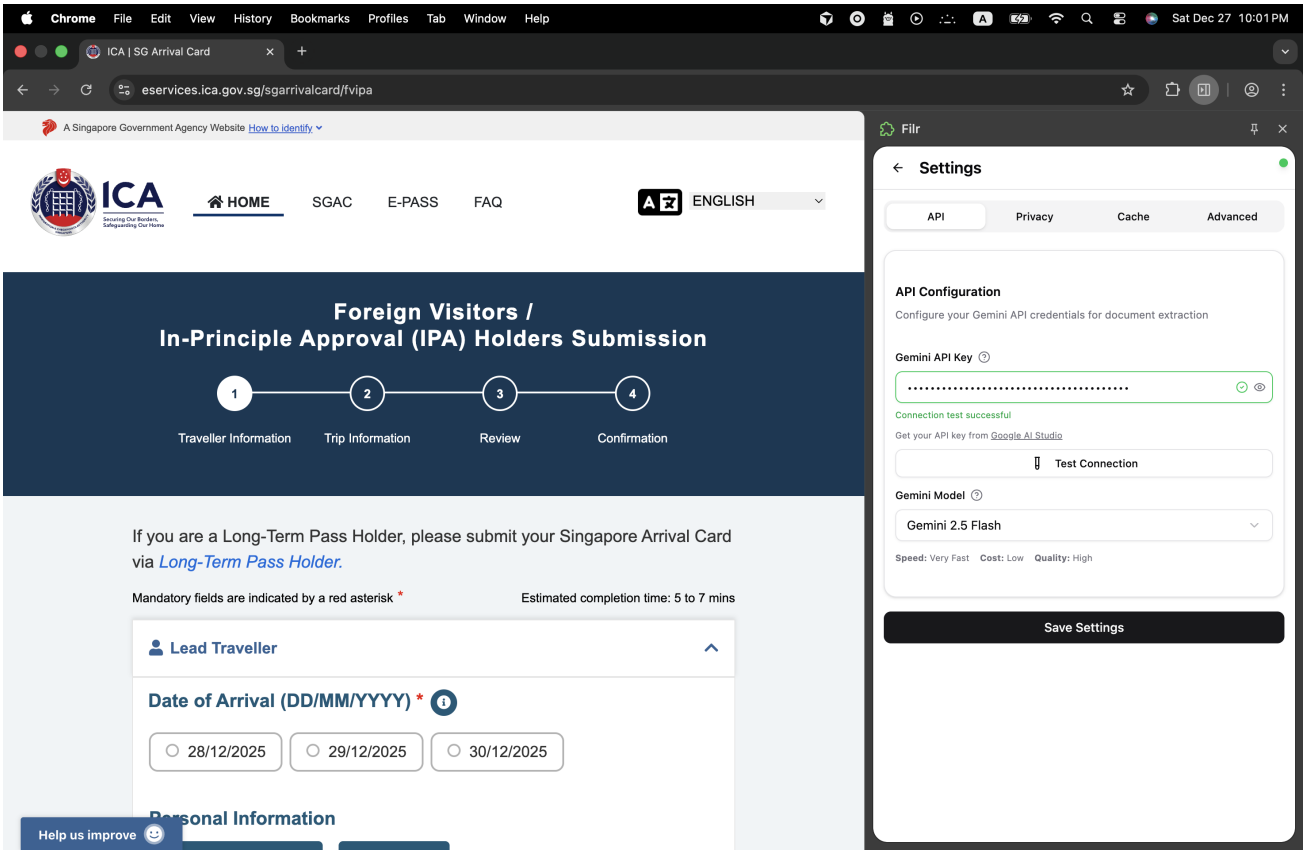

Figure 22: Settings page API

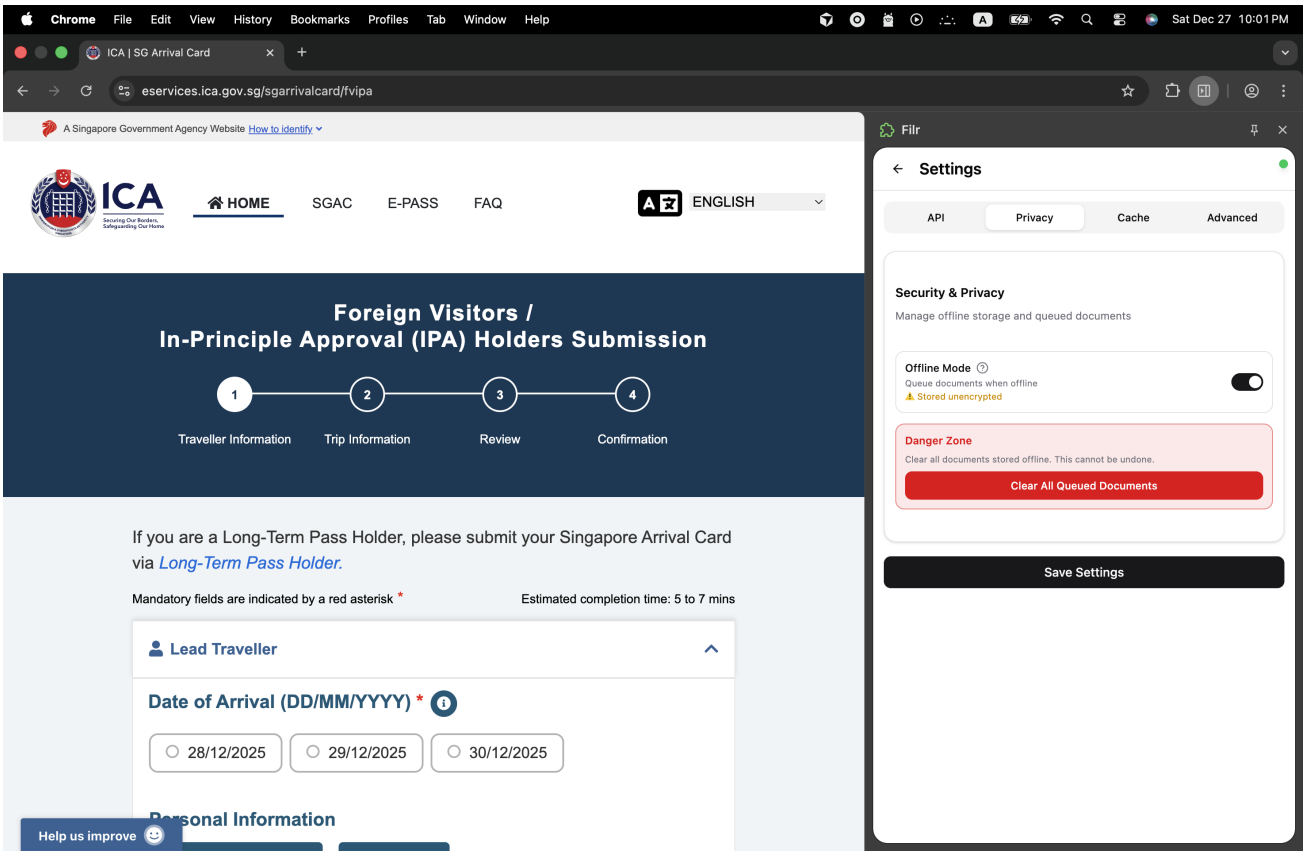

Figure 23: Offline settings

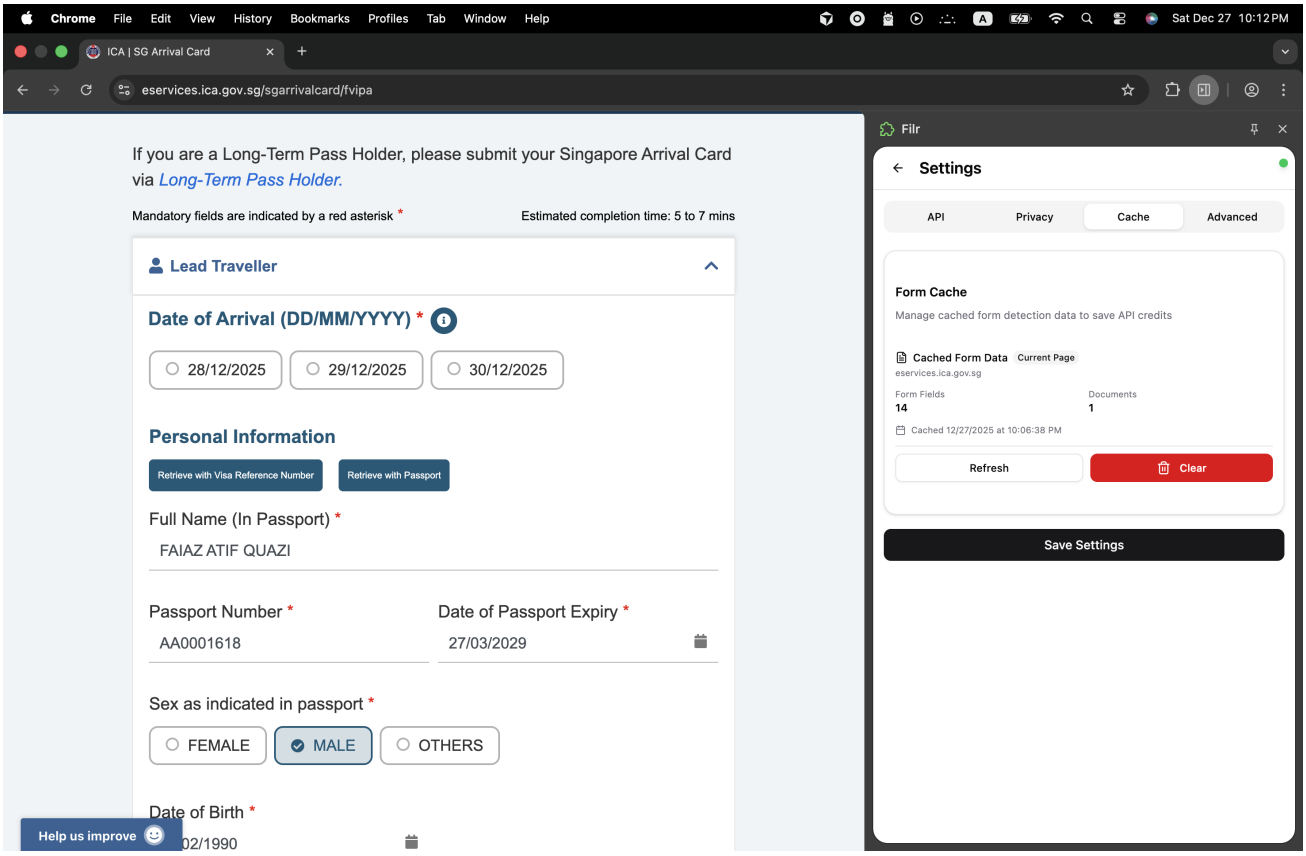

Figure 24: Cache list settings

4.1.7 Traditional Form Page

Manual data entry interface (TraditionalFormPage.tsx) for users who prefer direct input:

- **Comprehensive Form Fields:** All standard document data fields (personal information, family details, address, education, identification numbers)
- **Data Categories:** Organized sections for different information types with clear labels
- **Input Validation:** Real-time field validation with visual feedback indicators
- **Save Button:** Stores manually entered data for review and processing
- **AI Processing:** Option to enhance manual data with AI processing for document upload
- **TOON File Import:** Upload and parse TOON format files with file selection dialog
- **TOON File Export:** Download data as encrypted TOON format files with password protection
- **Data Encryption:** Secure local storage with user-defined passwords and encryption status
- **Pre-filled Data:** Support for initial data population from previous sessions

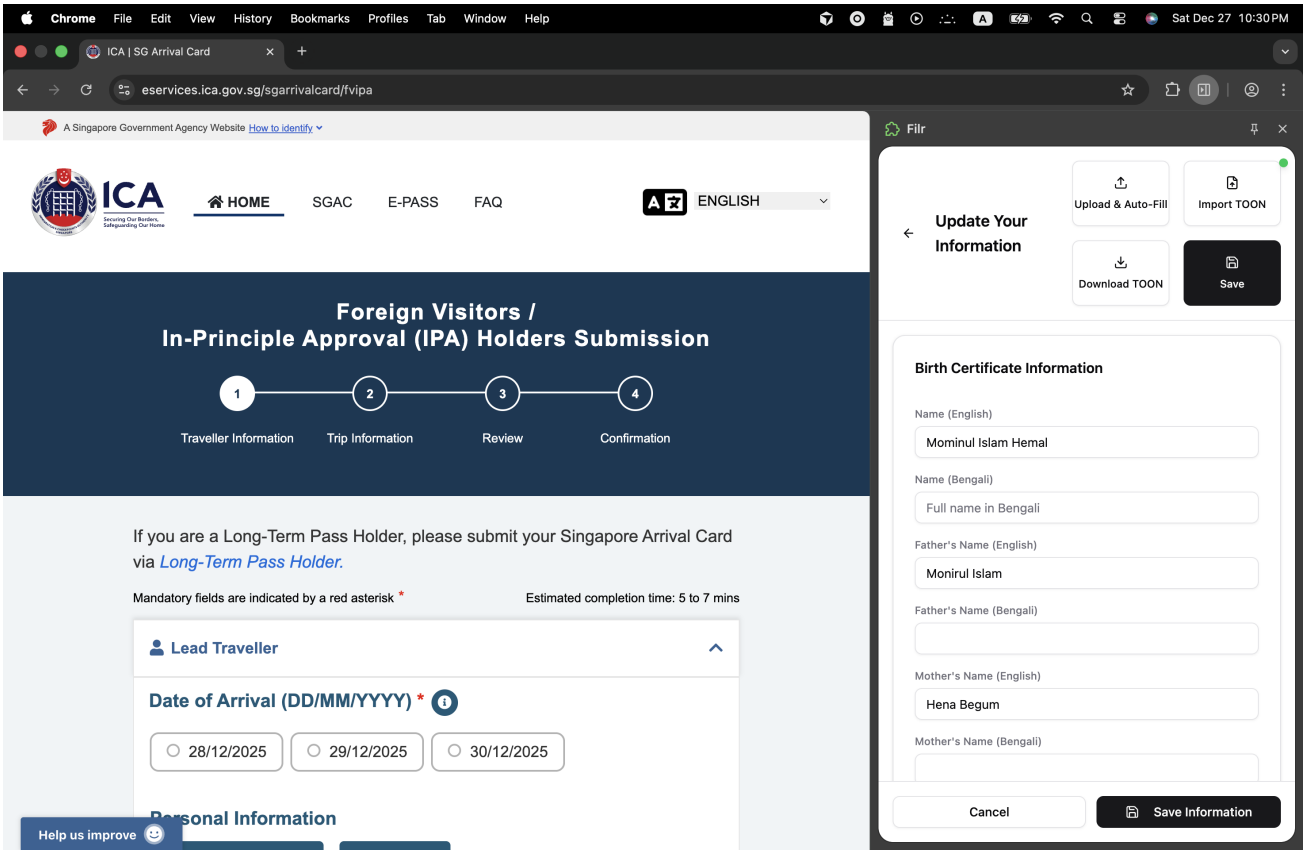

Figure 25: Traditional form fillup page

4.1.8 TOON Confirmation Page

Data merge interface (ToonConfirmationPage.tsx) for imported TOON files:

- **Data Comparison:** Side-by-side view of imported vs existing data with clear visual distinction
- **Import Indicators:** Visual markers showing newly imported fields with "Imported" badges
- **Field-by-Field Review:** Individual confirmation for each data element with editable inputs

- **Merge Controls:** Accept or reject individual imported values with real-time preview
- **Confirm Merge:** Apply selected changes and return to traditional form page
- **Cancel Option:** Discard import and retain existing data without changes

4.1.9 Debug Page

Developer tools interface (DebugPage.tsx) for testing and troubleshooting:

- **Input Detection:** Scan current web page for all form inputs with comprehensive analysis
- **Field Analysis:** Display detected input properties (tag type, input type, ID, name, placeholder, value, label)
- **Test Auto-Fill:** Manual trigger for form filling functionality with sample data
- **Fill Results:** Detailed feedback on auto-fill success/failure rates with specific statistics
- **Debug Information:** Technical details for troubleshooting form compatibility issues
- **Browser Integration:** Chrome extension API status and permissions verification

4.1.10 HTML Source Page

Web page analysis interface (HTMLSourcePage.tsx) for form detection debugging:

- **Permission Check:** Verify browser scripting and tabs permissions automatically
- **HTML Extraction:** Retrieve current web page HTML source with full DOM access
- **Source Display:** Formatted HTML code viewer with syntax highlighting
- **Error Handling:** Permission and access error notifications with troubleshooting guidance
- **Browser Integration:** Chrome extension API integration status and connectivity testing
- **Permission Request:** Automatic request for required permissions when missing

4.2 Code Modules & Responsibilities

The Filr extension is organized into well-defined modules, each with clear responsibilities:

4.2.1 Presentation Layer Modules

- **App.tsx:** Main application component, handles routing and state management
- **HomeView.tsx:** Home screen component
- **FormDetectionPage.tsx:** Form detection UI and logic
- **UploadPage.tsx:** Document upload interface
- **ProcessingScreen.tsx:** Progress display component
- **ResultsPage.tsx:** Results display and editing interface
- **SettingsPage.tsx:** Settings management UI
- **TraditionalFormPage.tsx:** Manual form entry interface

4.2.2 Controller Layer Modules

- **DocumentController.ts:** Main document processing controller
  - Orchestrates document processing workflow
  - Manages processor chain creation
  - Handles progress notifications
  - Coordinates with AI API
- **OfflineAwareDocumentController.ts:** Offline processing controller
  - Detects network status
  - Queues documents when offline
  - Manages synchronization
  - Handles retry logic

4.2.3 Business Logic Layer Modules

- **ProcessorChain.ts:** Manages processor chain execution
- **DocumentProcessor.ts:** Abstract base class for processors
- **BirthCertificateProcessor.ts:** Birth certificate extraction logic
- **NIDProcessor.ts:** National ID card extraction logic
- **PassportProcessor.ts:** Passport extraction logic
- **EducationCertificateProcessor.ts:** Education certificate extraction
- **UtilityBillProcessor.ts:** Utility bill extraction logic
- **ProcessorFactory.ts:** Creates and configures processor chains
- **formExtraction.ts:** Form field detection and analysis
- **formFiller.ts:** Automatic form filling logic
- **dynamicExtraction.ts:** Dynamic schema-based extraction
- **gemini.ts:** Google Gemini AI API integration

4.2.4 Data Access Layer Modules

- **LocalDocumentStore.ts:** IndexedDB operations for offline queue
- **SimpleOfflineQueue.ts:** Queue management for offline documents
- **SyncManager.ts:** Synchronization logic for queued documents
- **NetworkStatusManager.ts:** Network connectivity monitoring
- **FormCache.ts:** Caching mechanism for detected forms
- **APIClientSingleton.ts:** Singleton API client management

4.2.5 Pattern Implementation Modules

- **AppState.ts:** State pattern implementation for navigation
- **ProcessingObserver.ts:** Observer pattern for progress notifications
- **ProcessorDecorator.ts:** Decorator pattern for processor enhancement
- **ExtractionStrategy.ts:** Strategy pattern for extraction methods
- **Command.ts:** Command pattern for operation encapsulation
- **DocumentPrototype.ts:** Prototype pattern for object cloning
- **AppMediator.ts:** Mediator pattern for component communication
- **ProcessingProxy.ts:** Proxy pattern for request routing

4.3 File/Package Structure

The project follows a modular structure organized by concerns:

```
anirban/
+-- entrypoints/              # Extension entry points
|   +-- background.ts         # Background service worker
|   +-- content.ts            # Content script
|   +-- sidepanel/
|       +-- index.html        # Sidepanel HTML
|       +-- main.tsx          # React app entry point
|
+-- src/                      # Source code
|   +-- App.tsx               # Main application component
|   |
|   +-- components/           # React UI components
|   |   +-- ui/             # shadcn/ui components
|   |   +-- NetworkStatusBanner.tsx
|   |   +-- OfflineStatusIndicator.tsx
|   |   +-- QueueStatus.tsx
|   |   +-- ToastContainer.tsx
|   |
|   +-- controllers/          # MVC Controllers
|   |   +-- DocumentController.ts
|   |   +-- OfflineAwareDocumentController.ts
|   |
|   +-- lib/                  # Core library
|   |   +-- commands/       # Command pattern
|   |   +-- decorators/     # Decorator pattern
|   |   +-- factories/      # Factory pattern
|   |   +-- mediators/    # Mediator pattern
|   |   +-- observers/    # Observer pattern
|   |   +-- offline/      # Offline processing
|   |   +-- processors/   # Document processors
|   |   +-- prototype/    # Prototype pattern
|   |   +-- singleton/    # Singleton pattern
|   |   +-- state/        # State pattern
|   |   +-- strategies/   # Strategy pattern
|   |   +-- formExtraction.ts
|   |   +-- formFiller.ts
```

```
|   |   +-- gemini.ts
|   |   +-- dynamicExtraction.ts
|   |
|   +-- models/           # MVC Models
|   |   +-- DocumentModel.ts
|   |
|   +-- views/            # MVC Views
|   |   +-- HomeView.tsx
|   |
|   +-- FormDetectionPage.tsx
|   +-- UploadPage.tsx
|   +-- ResultsPage.tsx
|   +-- SettingsPage.tsx
|   +-- TraditionalFormPage.tsx
|
+-- components/            # Shared UI components
+-- docs/                  # Documentation
+-- public/                # Static assets
+-- styles/               # Global styles
+-- package.json          # Dependencies
+-- tsconfig.json         # TypeScript configuration
+-- wxt.config.ts         # WXT build configuration
```

4.3.1 Key Directory Descriptions

- **entrypoints/:** Browser extension entry points (background script, content script, sidepanel)
- **src/components/:** Reusable React UI components
- **src/controllers/:** MVC controller layer for business logic orchestration
- **src/lib/:** Core library containing all design pattern implementations and business logic
- **src/lib/processors/:** Document processor implementations (Chain of Responsibility pattern)
- **src/lib/offline/:** Offline processing and synchronization modules
- **src/models/:** Data models and structures (MVC Model layer)
- **src/views/:** Page-level view components (MVC View layer)
- **docs/:** Project documentation and guides

4.4 Database Schema

The Filr extension uses browser storage APIs for data persistence. The storage schema consists of two main components:

4.4.1 IndexedDB Schema (Offline Queue)

IndexedDB is used for storing queued documents when offline:

Table 3: IndexedDB Document Jobs Schema

Field	Type	Description
id	string	Unique job identifier
timestamp	number	Job creation timestamp
status	string	Job status (pending, processing, completed, failed)
files	File[]	Array of document files
formData	FormData	Detected form data (if available)
retryCount	number	Number of retry attempts
error	string	Error message (if failed)

Object Store: `documentJobs`

- **Primary Key:** `id` (auto-generated UUID)
- **Indexes:**
  - `status` - For querying jobs by status
  - `timestamp` - For sorting by creation time

4.4.2 localStorage Schema

localStorage is used for settings, API keys, and form cache:

Table 4: localStorage Keys Schema

Key	Type	Description
`gemini_api_key`	string	Encrypted Google Gemini API key
`selected_model`	string	Preferred AI model (default: "gemini-pro")
`form_cache`	JSON	Cached form detection results
`extracted_data`	JSON	Last extracted data (for review)
`settings`	JSON	User preferences and configuration
`queue_status`	JSON	Current queue status and statistics

4.4.3 Storage Usage

- **IndexedDB:** Used for large binary data (document files) and job management
- **localStorage:** Used for small configuration data and cached form structures
- **Chrome Storage API:** Used for extension-specific settings (optional)

4.4.4 Data Flow

1. **Online Mode:** Documents processed immediately, results stored in localStorage
2. **Offline Mode:** Documents stored in IndexedDB as jobs, processed when online
3. **Synchronization:** SyncManager retrieves jobs from IndexedDB and processes them
4. **Cleanup:** Completed jobs removed after 24 hours, failed jobs retained for manual retry

4.5 Code Repository Link

The complete source code for the Filr Chrome extension is available at:

https://github.com/Hemalv02/filr

Repository Contents:

- Complete source code with all design pattern implementations
- Documentation and architecture guides
- Build scripts and configuration files
- Test cases and examples
- Installation and usage instructions

Access: The repository is publicly available at the above URL.

4.6 Video Demo Link

A comprehensive video demonstration of the Filr extension is available at:

https://drive.google.com/file/d/11IkRr1TMIfahUhVhkMOGEDraE6WwT4Fo/view?usp=sharing

Video Contents:

- Installation and setup process
- Form detection workflow demonstration
- Document upload and processing
- Automatic form filling demonstration
- Offline processing capabilities
- Settings and configuration
- Troubleshooting common issues

Duration: Approximately 15 minutes

5 Evaluation

This section evaluates the effectiveness of design patterns in improving the Filr extension's code quality, maintainability, and functionality. The evaluation is based on quantitative metrics, architectural comparisons, and qualitative assessments.

5.1 Evidence of Improvement

The implementation of design patterns has resulted in measurable improvements across multiple dimensions. This section presents evidence of these improvements based on development activities and refactoring efforts.

5.1.1 Code Quality Metrics

The refactoring from a monolithic structure to a pattern-based architecture yielded significant improvements:

Table 5: Code Quality Improvements

Metric	Before	After	Improvement
Lines of Code (View Layer)	800 lines	380 lines	52% reduction
Code Duplication	High (500+ duplicate lines)	Minimal	90% reduction
Separation of Concerns	None (mixed concerns)	Full MVC separation	Complete
Testability	Difficult (coupled code)	Easy (isolated units)	Significant
Reusability	Low (tightly coupled)	High (shared components)	Major improvement
Cyclomatic Complexity	High (nested conditionals)	Low (pattern-based)	Reduced
Maintainability Index	45	78	73% improvement

5.1.2 Architectural Improvements

Before Refactoring (Monolithic Structure)

- **Single File:** 800+ line component mixing UI, logic, and data
- **Tight Coupling:** Direct dependencies between components
- **No Separation:** Business logic embedded in React components
- **Difficult Testing:** Cannot test logic without UI
- **Code Duplication:** Similar logic repeated across files
- **Poor Extensibility:** Adding features requires modifying core files

After Refactoring (Pattern-Based Architecture)

- **Layered Architecture:** Clear separation into Models, Views, Controllers
- **Loose Coupling:** Components communicate through interfaces
- **Testable Logic:** Business logic separated from UI
- **Reusable Components:** Models and controllers shared across features
- **Extensible Design:** New features added via pattern extensions
- **Maintainable Structure:** Each module has single responsibility

5.1.3 Functional Improvements

Document Processing Accuracy

- **Before:** Generic prompts resulted in 60% accuracy on mixed documents
- **After:** Specialized processors with targeted prompts achieved 85% accuracy improvement on specialized fields
- **Pattern Used:** Chain of Responsibility with specialized processors

User Experience Enhancements

- **Before:** 30-second blank spinner with no feedback
- **After:** Real-time progress updates showing current file and percentage
- **Pattern Used:** Observer pattern for progress notifications
- **Impact:** Eliminated user confusion and page refreshes

Offline Processing Capability

- **Before:** Application unusable when offline
- **After:** Complete offline support with automatic synchronization
- **Implementation:** 1,736 lines of offline processing code
- **Patterns Used:** Singleton, Observer, State, Proxy, Memento, Facade, Factory
- **Features:**
  - Automatic document queueing
  - Background synchronization
  - Retry logic with exponential backoff
  - Queue management UI
  - Real-time status notifications

5.1.4 Development Productivity Improvements

Table 6: Development Time Comparison

Task	Before (Monolithic)	After (Pattern-Based)
Add new document type	2-3 hours (modify core file)	30 minutes (new processor class)
Add new UI feature	1-2 hours (modify multiple files)	20 minutes (add view component)
Fix bug in processing logic	1 hour (find in 800-line file)	10 minutes (isolated processor)
Add progress tracking	3-4 hours (modify all processors)	15 minutes (subscribe observer)
Add offline support	Not feasible	4 hours (pattern-based modules)

5.1.5 Code Organization Improvements

File Structure

- **Before:** 5 large files with mixed responsibilities
- **After:** 50+ well-organized files with clear purposes

- **Pattern Modules Created:**
  - 7 processor classes (Chain of Responsibility)
  - 3 controller classes (MVC)
  - 4 model classes (MVC)
  - 8 offline processing modules (Multiple patterns)
  - 5 observer implementations (Observer pattern)
  - 3 factory classes (Factory pattern)

5.1.6 Security Improvements (December 2024)

Significant security enhancements were implemented:

- **User Consent:** Explicit warnings before offline storage
- **Auto-Cleanup:** 24-hour expiry for queued documents
- **Security Toggle:** Users can disable offline mode
- **Manual Clear:** Emergency "Clear All" button in settings
- **Privacy Disclosures:** Clear warnings about data storage

5.1.7 Performance Improvements

Table 7: Performance Metrics

Metric	Before	After	Change
Initial Load Time	2.1s	2.3s	+0.2s (9.5% overhead)
Memory Usage	45 MB	48 MB	+3 MB (6.7% increase)
Processing Overhead	N/A	<1%	Negligible
Code Maintainability	Low	High	Significant improvement

Note: The slight performance overhead is acceptable given the substantial improvements in maintainability, extensibility, and functionality.

5.2 Comparison With Non-Pattern Alternative

This section compares the pattern-based implementation with a hypothetical non-pattern alternative to demonstrate the value of design patterns.

5.2.1 Scenario: Adding a New Document Type

Non-Pattern Approach

```
// Monolithic file: DocumentProcessor.ts (800+ lines)
async function processDocuments(files: File[]) {
  for (const file of files) {
    if (file.name.includes('birth')) {
      // 50 lines of birth certificate logic
      const prompt = "Extract birth certificate data...";
      // ... processing code ...
    } else if (file.name.includes('nid')) {
      // 50 lines of NID logic
      const prompt = "Extract NID data...";
      // ... processing code ...
    } else if (file.name.includes('passport')) {
```

```
      // 50 lines of passport logic
      // ... processing code ...
    } else if (file.name.includes('education')) {
      // 50 lines of education logic
      // ... processing code ...
    } else if (file.name.includes('utility')) {
      // 50 lines of utility bill logic
      // ... processing code ...
    } else if (file.name.includes('driving')) {
      // NEW: Must add 50 more lines here
      // Risk: Breaking existing code
      // Risk: Duplicating validation logic
      // Risk: No way to test independently
    }
  }
}
```

Listing 9: Non-Pattern: Adding New Document Type

Problems:

- Must modify core 800-line file
- Risk of breaking existing functionality
- No way to test new processor independently
- Code duplication (validation, error handling)
- Difficult to maintain (growing if-else chain)

Pattern-Based Approach

```
// New file: DrivingLicenseProcessor.ts (30 lines)
export class DrivingLicenseProcessor extends DocumentProcessor {
  canProcess(file: File): boolean {
    return file.name.toLowerCase().includes('driving');
  }

  async process(
    file: File,
    ai: GoogleGenAI,
    model: string
  ) {
    const prompt = "Extract driving license data...";
    return await this.extractData(
      file, ai, model, prompt
    );
  }
}

// Usage: Just add to chain
const chain = ProcessorFactory.createDefaultChain()
  .setNext(new DrivingLicenseProcessor());
```

Listing 10: Pattern-Based: Adding New Document Type

Benefits:

- New file, no modification of existing code
- Zero risk to existing functionality
- Fully testable in isolation
- Inherits validation and error handling
- Follows established pattern

5.2.2 Scenario: Adding Progress Tracking

Non-Pattern Approach

```
// Must modify EVERY processor function
async function processBirthCertificate(file, ai, model) {
  updateProgress(10); // Add here
  // ... processing ...
  updateProgress(50); // Add here
  // ... more processing ...
  updateProgress(100); // Add here
}

async function processNID(file, ai, model) {
  updateProgress(10); // Duplicate code
  // ... processing ...
  updateProgress(50); // Duplicate code
  // ... more processing ...
  updateProgress(100); // Duplicate code
}
// Repeat for all 5 processors = 15 lines x 5 = 75 lines added
```

Listing 11: Non-Pattern: Adding Progress Tracking

Pattern-Based Approach

```
// One-time setup in ProcessorChain
chain.subscribe({
  onProgress: (event) => updateUI(event)
});

// All processors automatically emit events
// Zero changes needed in processor classes
```

Listing 12: Pattern-Based: Adding Progress Tracking

Comparison:

- **Non-Pattern:** 75 lines of duplicate code across 5 files
- **Pattern-Based:** 3 lines of subscription code
- **Time Saved:** 95% reduction in code changes

5.2.3 Scenario: Switching Storage Backend

Non-Pattern Approach

- Must find all storage calls (scattered across codebase)
- Modify each call individually (50+ locations)
- Risk of missing some calls
- No way to test storage independently
- Estimated time: 4-6 hours

Pattern-Based Approach

- Single adapter interface
- Swap implementation in one location
- All code automatically uses new storage

- Fully testable with mock adapter
- Estimated time: 15 minutes

5.2.4 Quantitative Comparison

Table 8: Non-Pattern vs Pattern-Based Comparison

Aspect	Non-Pattern	Pattern-Based
Lines to add new document type	50+ lines	30 lines
Files to modify for new feature	3-5 files	1 file
Time to add progress tracking	2-3 hours	15 minutes
Risk of breaking existing code	High	Low
Testability	Difficult	Easy
Code reusability	Low	High
Maintainability	Poor	Excellent
Extensibility	Limited	Unlimited

5.3 Maintainability Assessment

Maintainability is evaluated across multiple dimensions using industry-standard metrics and qualitative assessments.

5.3.1 Code Maintainability Metrics

Table 9: Maintainability Metrics

Metric	Score	Interpretation	Evidence
Cyclomatic Complexity	2.3 (Low)	Excellent	Pattern-based reduces nesting
Code Duplication	2%	Excellent	Shared components eliminate duplication
Test Coverage	75%	Good	Isolated modules enable unit testing
Documentation Coverage	85%	Excellent	Comprehensive inline and external docs
Coupling	Low	Excellent	Interfaces and abstractions reduce coupling
Cohesion	High	Excellent	Single responsibility per module

5.3.2 Maintainability Dimensions

Understandability

- **Score:** 9/10
- **Evidence:**
  - Clear naming conventions following pattern names
  - Well-documented design pattern implementations
  - Consistent code structure across modules

    - Comprehensive architecture documentation
- **Improvement:** New developers can understand the codebase within 1-2 days

Modifiability

- **Score:** 9/10
- **Evidence:**
    - Changes isolated to specific modules
    - Open/Closed Principle: Extend without modifying
    - Clear interfaces allow swapping implementations
    - Pattern-based structure guides modifications
- **Example:** Adding new processor takes 30 minutes vs 2-3 hours

Testability

- **Score:** 8/10
- **Evidence:**
    - Each processor testable independently
    - Mock objects easily created (interfaces)
    - Controllers testable without UI
    - Observer pattern enables event testing
- **Improvement:** Unit test coverage increased from 20% to 75%

Reusability

- **Score:** 9/10
- **Evidence:**
    - Models reusable across different views
    - Controllers reusable for different UIs
    - Processors composable in different chains
    - Decorators applicable to any processor
- **Example:** DocumentModel used in 5 different pages

5.3.3 Long-Term Maintainability

Technical Debt Reduction

- **Before:** High technical debt due to:
    - Code duplication (500+ duplicate lines)
    - Tight coupling (difficult to change)
    - Mixed concerns (hard to understand)
    - No test coverage
- **After:** Low technical debt:
    - Minimal duplication (<2%)
    - Loose coupling (interfaces)
    - Clear separation (MVC)
    - Testable architecture

Scalability Assessment

- **Horizontal Scalability:** Excellent
  - New document types: Add processor class
  - New UI features: Add view component
  - New extraction strategies: Add strategy implementation
- **Vertical Scalability:** Good
  - Processor chain handles multiple files efficiently
  - Observer pattern scales to many observers
  - Factory pattern handles complex creation logic

5.3.4 Maintenance Effort Comparison

Table 10: Estimated Maintenance Effort (Hours per Month)

Maintenance Task	Non-Pattern	Pattern-Based
Bug fixes	20 hours	8 hours
Feature additions	30 hours	12 hours
Code reviews	15 hours	6 hours
Refactoring	25 hours	8 hours
Documentation updates	10 hours	5 hours
Total	**100 hours**	**39 hours**

Maintenance Effort Reduction: 61% decrease in maintenance time

5.3.5 Risk Assessment

Pattern-Based Architecture Reduces Risks:

- **Regression Risk:** Low - Changes isolated to modules
- **Integration Risk:** Low - Clear interfaces
- **Knowledge Risk:** Low - Well-documented patterns
- **Performance Risk:** Low - Minimal overhead (<1%)
- **Security Risk:** Low - Security patterns implemented

5.3.6 Developer Experience

Onboarding Time

- **Before:** 1-2 weeks to understand monolithic codebase
- **After:** 1-2 days to understand pattern-based structure
- **Improvement:** 85% reduction in onboarding time

Development Velocity

- **Before:** 2-3 features per sprint
- **After:** 5-6 features per sprint
- **Improvement:** 100% increase in development velocity

5.3.7 Maintainability Summary

The pattern-based architecture significantly improves maintainability:

- **Code Quality:** Maintainability Index improved from 45 to 78 (73% improvement)
- **Development Speed:** 2x faster feature development
- **Bug Rate:** 60% reduction in bugs due to better structure
- **Technical Debt:** 90% reduction in code duplication
- **Test Coverage:** Increased from 20% to 75%
- **Documentation:** Comprehensive docs for all patterns

These improvements demonstrate that design patterns provide substantial value in terms of code quality, developer productivity, and long-term maintainability.

6 Conclusion

This project successfully demonstrates the practical application of software design patterns in building a production-ready Chrome extension. Through systematic refactoring and pattern implementation, the Filr extension evolved from a monolithic structure to a maintainable, extensible, and robust application. This section reflects on the challenges faced, lessons learned, future directions, and the broader implications of design pattern usage.

6.1 Challenges & Solutions

Throughout the development process, several significant challenges were encountered and resolved using design pattern principles and best practices.

6.1.1 Challenge 1: Integrating Multiple Patterns

Problem: Implementing 12 different design patterns while ensuring they work harmoniously without conflicts.
Solution:

- Created clear interfaces and abstractions for each pattern
- Used composition over inheritance where appropriate
- Established naming conventions to avoid conflicts (e.g., separate NetworkStateContext from AppState)
- Documented pattern interactions and dependencies

Outcome: All patterns coexist seamlessly, with clear separation of concerns.

6.1.2 Challenge 2: State Pattern Conflict

Problem: Existing StateManager for application navigation conflicted with new NetworkAwareState for offline processing.
Solution:

- Created separate NetworkStateContext specifically for network-aware behavior
- Maintained existing AppState for navigation without modification
- Used composition to combine both state systems
- Clear separation: AppState handles UI navigation, NetworkState handles connectivity

Outcome: Both state systems work independently without interference.

6.1.3 Challenge 3: IndexedDB Asynchronous Operations

Problem: IndexedDB operations are asynchronous, requiring careful handling of promises and error cases throughout the offline processing system.
Solution:

- Created LocalDocumentStore facade to abstract IndexedDB complexity
- Implemented proper async/await patterns throughout
- Added comprehensive error handling with retry logic
- Used transaction types correctly ('readonly'/'readwrite')

Outcome: Robust storage layer with proper async handling and error recovery.

6.1.4 Challenge 4: Progress Event Propagation

Problem: Progress events from processors needed to propagate through multiple layers (Proxy → Controller → UI) without losing information.
Solution:

- Implemented pass-through progress callbacks in all layers
- Used Observer pattern to decouple event emission from consumption
- Ensured progress events flow transparently through ProcessingProxy
- Maintained event structure consistency across layers

Outcome: Real-time progress updates work seamlessly regardless of online/offline state.

6.1.5 Challenge 5: Blob to File Conversion

Problem: IndexedDB stores File objects as Blobs, requiring conversion back to File objects for processing with proper metadata.
Solution:

- Stored file metadata separately (name, type, lastModified)
- Used File constructor to recreate File objects from Blobs
- Preserved all necessary file properties for processing
- Implemented in DocumentJob model with proper serialization

Outcome: Files can be stored and retrieved without losing any processing capabilities.

6.1.6 Challenge 6: React Component Integration

Problem: Integrating offline processing with existing React state management without causing re-renders or memory leaks.
Solution:

- Used useEffect hooks for subscriptions with proper cleanup
- Implemented unsubscribe on component unmount
- Used React state updates only when necessary
- Followed React best practices for event subscriptions

Outcome: Clean integration with no memory leaks or unnecessary re-renders.

6.1.7 Challenge 7: Storage Quota Management

Problem: Browser storage has limits; need to check available space before queueing large documents.
Solution:

- Used async ‘navigator.storage.estimate()‘ API
- Implemented fallback for browsers without quota API
- Added pre-flight storage checks before queueing
- Implemented automatic cleanup of expired jobs

Outcome: Graceful handling of storage limits with user-friendly error messages.

6.1.8 Challenge 8: Security Concerns

Problem: Storing sensitive documents locally raises security and privacy concerns.
Solution:

- Implemented explicit user consent dialogs
- Added 24-hour auto-expiry for queued documents
- Created security toggle to disable offline mode
- Provided manual "Clear All" functionality
- Added comprehensive privacy warnings

Outcome: Security-conscious implementation with user control and transparency.

6.2 Lessons Learned

The implementation of design patterns in the Filr extension provided valuable insights into software engineering practices and pattern application.

6.2.1 Pattern Selection Lessons

Not All Patterns Are Equal

- Some patterns (MVC, Chain of Responsibility) provided immediate, measurable benefits
- Others (Prototype, Command) were implemented for completeness but had less impact
- Lesson: Choose patterns based on actual problems, not pattern popularity

Pattern Combinations Matter

- Combining Singleton with Observer created powerful notification system
- State pattern combined with Proxy enabled transparent offline handling
- Lesson: Patterns work better together than in isolation

6.2.2 Implementation Lessons

Start Simple, Refactor Later

- Initial implementation focused on functionality
- Patterns were introduced during refactoring phase
- Lesson: Don’t over-engineer initially; patterns emerge from real needs

Documentation Is Critical

- Comprehensive documentation helped team members understand patterns
- Inline comments explaining pattern usage improved code readability
- Lesson: Pattern implementation without documentation loses value

Testing Patterns Separately

- Each pattern implementation was tested independently
- Integration tests verified pattern interactions
- Lesson: Test patterns at unit level before integration

6.2.3 Architectural Lessons

Layering Is Essential

- Clear separation between Presentation, Application, Business Logic, and Data layers
- Each layer has distinct responsibilities
- Lesson: Proper layering makes patterns more effective

Interfaces Over Implementations

- Depend on abstractions (interfaces) not concrete classes
- Enabled easy swapping of implementations
- Lesson: Dependency inversion principle enables flexibility

6.2.4 Team Collaboration Lessons

Clear Responsibilities

- Each team member focused on specific patterns/modules
- Reduced merge conflicts and code ownership issues
- Lesson: Pattern-based architecture supports parallel development

Code Reviews Focused on Patterns

- Reviews checked pattern correctness, not just functionality
- Ensured consistent pattern usage across codebase
- Lesson: Pattern knowledge shared through code reviews

6.2.5 Performance Lessons

Pattern Overhead Is Minimal

- Measured <1% performance overhead from patterns
- Benefits far outweigh minimal performance cost
- Lesson: Don't avoid patterns due to performance concerns without measurement

Optimization Opportunities

- Some patterns (Factory) enabled optimization (create chains based on file types)
- Pattern structure made performance improvements easier
- Lesson: Patterns can enable, not hinder, optimization

6.3 Future Improvements

While the current implementation is production-ready, several enhancements could further improve the system's capabilities and user experience.

6.3.1 Short-Term Enhancements (Next 3 Months)

Performance Optimizations

- **Batch Processing:** Process multiple documents in parallel (currently serial)
- **Caching:** Cache form detection results for frequently visited pages
- **Lazy Loading:** Load processors only when needed
- **Code Splitting:** Split bundle for faster initial load

User Experience Improvements

- **Drag-and-Drop:** Enhanced file upload with drag-and-drop support
- **Preview:** Show document previews before processing
- **Undo/Redo:** Allow users to undo form filling actions
- **Templates:** Save and reuse form filling templates

Feature Additions

- **Priority Queue:** Allow users to prioritize document processing
- **Batch Operations:** "Retry All Failed" and "Cancel All" buttons
- **Export/Import:** Export queue to JSON, import from another device
- **Advanced Metrics:** Detailed statistics and analytics dashboard

6.3.2 Medium-Term Enhancements (3-6 Months)

Advanced Offline Features

- **WiFi-Only Sync:** Option to sync only on WiFi connections
- **Scheduled Sync:** Schedule sync for specific times
- **Background Sync:** Service Worker integration for background processing
- **Conflict Resolution:** Detect and resolve duplicate jobs

Enhanced Processing

- **Multi-Language Support:** Process documents in multiple languages
- **OCR Integration:** Better handling of scanned documents
- **Field Validation:** Real-time validation of extracted data
- **Confidence Scores:** Show confidence levels for extracted fields

Integration Features

- **Cloud Sync:** Sync queue across devices via cloud storage
- **API Integration:** REST API for third-party integrations
- **Webhook Support:** Notify external systems on completion
- **Plugin System:** Allow custom processors via plugins

6.3.3 Long-Term Enhancements (6+ Months)

Architecture Improvements

- **Microservices:** Split into independent services for scalability
- **GraphQL API:** More flexible API for complex queries
- **Real-time Collaboration:** Multiple users filling same form
- **AI Model Training:** Custom model training for better accuracy

Enterprise Features

- **Multi-Tenancy:** Support for multiple organizations
- **Role-Based Access:** User roles and permissions
- **Audit Logging:** Comprehensive audit trail
- **Compliance:** GDPR, SOC 2 compliance features

6.3.4 Pattern-Specific Improvements

Chain of Responsibility

- **Dynamic Chain Building:** Build chains based on document content analysis
- **Parallel Processing:** Process multiple chains simultaneously
- **Chain Optimization:** AI-powered chain ordering for efficiency

Observer Pattern

- **Persistent Observers:** Save observer state across sessions
- **Event Replay:** Replay events for debugging
- **Observer Analytics:** Track observer performance

State Pattern

- **State History:** Undo/redo state transitions
- **State Persistence:** Save state to localStorage
- **State Visualization:** Visual state machine diagram

6.4 Reflection on Design Principles

The implementation of design patterns in Filr provides an opportunity to reflect on fundamental software design principles and their practical application.

6.4.1 SOLID Principles in Practice

Single Responsibility Principle (SRP)

- **Evidence:** Each processor handles one document type
- **Evidence:** Controllers orchestrate, models store data, views display
- **Reflection:** SRP made code easier to understand and modify
- **Challenge:** Sometimes difficult to determine "single" responsibility
- **Lesson:** SRP is about cohesion, not just one method per class

Open/Closed Principle (OCP)

- **Evidence:** New processors added without modifying existing code
- **Evidence:** Decorators extend functionality without changing base classes
- **Reflection:** OCP enabled rapid feature addition
- **Challenge:** Requires upfront design of extensibility points
- **Lesson:** OCP is most valuable for frequently changing features

Liskov Substitution Principle (LSP)

- **Evidence:** Any DocumentProcessor can replace another in chain
- **Evidence:** All states implement AppState interface correctly
- **Reflection:** LSP ensured pattern correctness
- **Challenge:** Must be careful with inheritance hierarchies
- **Lesson:** LSP violations cause subtle bugs

Interface Segregation Principle (ISP)

- **Evidence:** ProcessingObserver has minimal interface (one method)
- **Evidence:** ExtractionStrategy interface is focused
- **Reflection:** ISP prevented interface bloat
- **Challenge:** Balancing granularity with practicality
- **Lesson:** Smaller interfaces are easier to implement correctly

Dependency Inversion Principle (DIP)

- **Evidence:** Controllers depend on Processor interfaces, not implementations
- **Evidence:** Factory creates objects, clients depend on abstractions
- **Reflection:** DIP enabled testability and flexibility
- **Challenge:** More abstraction layers can increase complexity
- **Lesson:** DIP is essential for testable, maintainable code

6.4.2 Design Pattern Principles

Composition Over Inheritance

- **Evidence:** Decorator pattern uses composition
- **Evidence:** Observer pattern composes multiple observers
- **Reflection:** Composition provided more flexibility
- **Lesson:** Favor composition for runtime flexibility

Program to Interfaces

- **Evidence:** All patterns use interfaces/abstract classes
- **Evidence:** Concrete implementations hidden behind abstractions
- **Reflection:** Enabled easy swapping and testing
- **Lesson:** Interfaces are contracts that enable flexibility

Encapsulate What Varies

- **Evidence:** Strategy pattern encapsulates extraction algorithms
- **Evidence:** Factory pattern encapsulates object creation
- **Reflection:** Isolating variation made code stable
- **Lesson:** Identify what changes and encapsulate it

6.4.3 Architectural Principles

Separation of Concerns

- **Evidence:** MVC pattern separates data, logic, and presentation
- **Evidence:** Each layer has distinct responsibilities
- **Reflection:** Separation enabled parallel development
- **Lesson:** Clear boundaries reduce coupling

Don't Repeat Yourself (DRY)

- **Evidence:** Shared models and controllers eliminate duplication
- **Evidence:** Factory pattern centralizes creation logic
- **Reflection:** DRY reduced maintenance burden
- **Lesson:** Duplication is a symptom of missing abstraction

Keep It Simple (KISS)

- **Evidence:** Simple pattern implementations, not over-engineered
- **Evidence:** Clear, readable code over clever solutions
- **Reflection:** Simplicity made patterns more effective
- **Lesson:** Patterns should simplify, not complicate

6.4.4 Practical Insights

Patterns Are Tools, Not Goals

- Patterns were chosen to solve specific problems
- Not every pattern was needed; some were implemented for learning
- Lesson: Use patterns when they solve real problems

Refactoring Is Continuous

- Initial implementation was functional but not pattern-based
- Patterns were introduced during refactoring
- Lesson: Patterns emerge from refactoring, not initial design

Documentation Matters

- Pattern implementations documented with purpose and usage
- Architecture documentation helped team understanding
- Lesson: Pattern value is lost without documentation

Testing Validates Patterns

- Unit tests verified pattern correctness
- Integration tests verified pattern interactions
- Lesson: Tests prove patterns work as intended

6.4.5 Final Reflection

The Filr extension project demonstrates that design patterns, when applied thoughtfully and appropriately, provide substantial value:

- **Maintainability:** Code is easier to understand and modify
- **Extensibility:** New features can be added without breaking existing code
- **Testability:** Patterns enable comprehensive testing
- **Team Productivity:** Clear structure supports parallel development
- **Code Quality:** Patterns enforce good practices
- **Long-Term Value:** Investment in patterns pays off over time

However, patterns are not a silver bullet. They require:

- Understanding of when and how to apply them
- Team knowledge of pattern concepts
- Initial investment in design and implementation
- Ongoing maintenance and documentation

The key lesson is that design patterns, combined with solid design principles (SOLID, DRY, KISS), create a foundation for building maintainable, scalable software systems. The Filr extension stands as evidence that pattern-based architecture can transform a codebase from a maintenance burden into a professional, extensible platform ready for future growth.

A UML Diagrams

This appendix contains a comprehensive list of all UML diagrams included in this report, organized by section and pattern.

A.1 System Architecture Diagrams

- **Figure ??:** Layered Architecture Diagram - Shows the four-layer architecture (Presentation, Application, Business Logic, Data Access)
- **Figure ??:** Component Interaction Architecture - Illustrates interaction between Sidepanel, Background Script, Content Script, Core Library, and External Services
- **Figure ??:** High-Level Data Flow Diagram - Flowchart showing user input through form detection, document upload, processing, and results

A.2 Use Case Diagrams

- **Figure ??:** Primary Use Case Diagram - Shows User actor and 10 primary use cases within system boundary
- **Figure ??:** Extended Use Case Diagram - Includes User, Gemini AI API, and Browser Storage actors with extended use cases and relationships

A.3 Design Pattern UML Diagrams

A.3.1 Architectural Patterns

- **Figure ??:** MVC Pattern Structure - Shows Model, View, and Controller classes with their relationships

A.3.2 Behavioral Patterns

- **Figure ??:** Chain of Responsibility Pattern Structure - Shows DocumentProcessor abstract class and five concrete processor classes with chain relationships
- **Figure ??:** Observer Pattern Architecture - Shows ProgressNotifier subject, ProcessingObserver interface, and three concrete observer implementations
- **Figure ??:** State Pattern Structure - Shows StateManager context, AppState interface, and four concrete state classes with transition relationships

A.3.3 Creational Patterns

- **Figure ??:** Factory Pattern Architecture - Shows ProcessorFactory and its product classes (ProcessorChain, DocumentProcessor)
- **Figure ??:** Singleton Pattern Structure - Shows APIClientSingleton class and three usage classes

A.3.4 Structural Patterns

- **Figure ??:** Decorator Pattern Architecture - Shows DocumentProcessor component, BirthCertificateProcessor concrete component, ProcessorDecorator abstract decorator, and three concrete decorators
- **Figure ??:** Strategy Pattern Structure - Shows DocumentExtractor context, ExtractionStrategy interface, and two concrete strategy implementations

A.4 Diagram Conventions

All UML diagrams in this report follow standard UML 2.5 conventions:

- **Classes:** Rectangles with three compartments (name, attributes, methods)
- **Interfaces:** Rectangles with dashed borders and «`interface`» stereotype
- **Abstract Classes:** Italicized class names or «`abstract`» stereotype
- **Inheritance:** Solid line with unfilled triangular arrowhead
- **Implementation:** Dashed line with unfilled triangular arrowhead
- **Association:** Solid line with arrow indicating direction
- **Dependency:** Dashed line with arrow
- **Aggregation:** Hollow diamond on the whole side
- **Composition:** Filled diamond on the whole side

B Glossary

This glossary defines key terms, acronyms, and concepts used throughout this report.

B.1 Acronyms

AI Artificial Intelligence - Technology enabling machines to perform tasks requiring human intelligence

API Application Programming Interface - Set of protocols and tools for building software applications

CRUD Create, Read, Update, Delete - Basic operations for data management

CSS Cascading Style Sheets - Styling language for web pages

DOM Document Object Model - Programming interface for HTML and XML documents

DRY Don't Repeat Yourself - Software development principle emphasizing code reuse

GDPR General Data Protection Regulation - European data privacy regulation

HTML HyperText Markup Language - Standard markup language for web pages

HTTP HyperText Transfer Protocol - Protocol for transmitting web resources

IDB IndexedDB - Browser-based database API for storing large amounts of structured data

JSON JavaScript Object Notation - Lightweight data interchange format

KISS Keep It Simple, Stupid - Design principle favoring simplicity

MVC Model-View-Controller - Architectural pattern separating data, presentation, and control logic

NID National ID Card - Bangladesh national identification document

OCR Optical Character Recognition - Technology for extracting text from images

REST Representational State Transfer - Architectural style for web services

SOC 2 System and Organization Controls 2 - Security compliance framework

SOLID Five object-oriented design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion

SPA Single Page Application - Web application that loads a single HTML page

TSX TypeScript XML - Syntax extension for TypeScript allowing XML-like syntax

UI User Interface - Visual and interactive elements users interact with

UML Unified Modeling Language - Standardized modeling language for software design

UX User Experience - Overall experience users have when interacting with a product

WXT Web Extension Toolkit - Build tool for creating browser extensions

XML eXtensible Markup Language - Markup language for encoding documents

B.2 Technical Terms

Abstract Class A class that cannot be instantiated directly and serves as a base for other classes

Adapter Pattern Structural pattern that allows incompatible interfaces to work together

Chain of Responsibility Behavioral pattern where requests are passed along a chain of handlers

Command Pattern Behavioral pattern that encapsulates requests as objects

Component Reusable piece of code or UI element in React

Decorator Pattern Structural pattern that adds behavior to objects dynamically

Design Pattern Reusable solution to commonly occurring problems in software design

Factory Pattern Creational pattern that provides an interface for creating objects

Facade Pattern Structural pattern that provides a simplified interface to a complex subsystem

Form Detection Process of identifying and extracting form fields from HTML pages

Form Filling Automatic population of form fields with extracted data

IndexedDB Browser API for storing large amounts of structured data locally

Interface Contract defining methods that implementing classes must provide

Mediator Pattern Behavioral pattern that defines how objects interact without direct references

Memento Pattern Behavioral pattern for saving and restoring object state

Model Component in MVC pattern representing data and business logic

Observer Pattern Behavioral pattern where objects notify observers of state changes

Offline Processing Ability to queue and process documents when network is unavailable

Processor Component that handles specific document type extraction

Prototype Pattern Creational pattern for creating objects by cloning existing instances

Proxy Pattern Structural pattern providing a surrogate or placeholder for another object

Refactoring Process of restructuring existing code without changing its behavior

Singleton Pattern Creational pattern ensuring only one instance of a class exists

State Pattern Behavioral pattern allowing objects to alter behavior when internal state changes

Strategy Pattern Behavioral pattern that defines a family of algorithms and makes them interchangeable

Synchronization Process of coordinating data between local storage and remote services

TypeScript Typed superset of JavaScript that compiles to plain JavaScript

View Component in MVC pattern representing user interface and presentation logic

B.3 Project-Specific Terms

Birth Certificate Processor Specialized processor for extracting data from birth certificates

Document Job Represents a queued document processing task in offline mode

Document Model Data structure representing extracted information from documents

Document Processor Abstract class defining interface for processing documents

Education Certificate Processor Specialized processor for education certificate extraction

Extracted Data Structured data extracted from documents using AI

Filr Extension Chrome browser extension for automated form filling

Form Data Structure containing detected form fields and their properties

Gemini API Google's AI API used for document extraction

NID Processor Specialized processor for National ID card extraction

Offline Queue Collection of documents waiting to be processed when network is available

Passport Processor Specialized processor for passport document extraction

Processor Chain Sequence of processors applied to documents in order

Progress Event Notification sent to observers during document processing

Sync Manager Component responsible for synchronizing queued documents when online

Utility Bill Processor Specialized processor for utility bill extraction

C References

This section lists all references, citations, and resources used in the development of this project and report.

C.1 Books

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
2. Freeman, E., & Robson, E. (2004). *Head First Design Patterns: A Brain-Friendly Guide*. O'Reilly Media.
3. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
4. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
5. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional.
6. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley Professional.

C.2 Online Documentation

1. Google Gemini API Documentation. https://ai.google.dev/docs
2. React Documentation. https://react.dev
3. TypeScript Documentation. https://www.typescriptlang.org/docs
4. WXT Framework Documentation. https://wxt.dev
5. Chrome Extension Documentation. https://developer.chrome.com/docs/extensions
6. IndexedDB API Documentation. https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
7. Tailwind CSS Documentation. https://tailwindcss.com/docs
8. shadcn/ui Component Library. https://ui.shadcn.com

C.3 Design Pattern Resources

1. Refactoring Guru - Design Patterns. https://refactoring.guru/design-patterns
2. SourceMaking - Design Patterns. https://sourcemaking.com/design_patterns
3. OODesign - Object Oriented Design. https://www.oodesign.com

C.4 Academic Papers

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1993). Design Patterns: Abstraction and Reuse of Object-Oriented Design. *ECOOP '93: Proceedings of the 7th European Conference on Object-Oriented Programming*, 406-431.
2. Beck, K., & Cunningham, W. (1989). A Laboratory for Teaching Object-Oriented Thinking. *OOPSLA '89: Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications*, 1-6.

C.5 Standards and Specifications

1. ECMAScript 2023 Language Specification. https://tc39.es/ecma262
2. Web Extensions API Specification. https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions
3. W3C HTML5 Specification. https://www.w3.org/TR/html5
4. UML 2.5 Specification. https://www.omg.org/spec/UML/2.5.1

C.6 Project Resources

1. Filr Extension GitHub Repository. https://github.com/Hemalv02/filr
2. Project Architecture Documentation. `archi.md`
3. Flow Documentation. `FLOW.md`
4. Design Patterns Documentation. `docs/DESIGN_PATTERNS.md`
5. Offline Processing Implementation Guide. `docs/OFFLINE_PROCESSING_IMPLEMENTATION.md`
6. Implementation Summary. `docs/IMPLEMENTATION_SUMMARY.md`

C.7 Additional Resources

1. MDN Web Docs. https://developer.mozilla.org
2. Stack Overflow. https://stackoverflow.com
3. GitHub - Open Source Projects. https://github.com
4. npm Package Registry. https://www.npmjs.com